

KUMARAGURU COLLEGE OF TECHNOLOGY

DEPARTMENT OF ARTIFICIAL INTELGINCE AND DATA SCIENCE

24AD1204 – DATA SCIENCE AND VISULIZATION

NAME	ROLLNO
GOKULNAATH M	24BAD026
JAYARAKSHA REGURAJ	24BAD044
KAMALESH N	24BAD054
NISHANTH P	24BAD405

Week No: 1**Team Formation & Data Hunting****Objectives:**

Selection of dataset. Setting up GitHub. Introduction to Pandas/NumPy.

Team Composition:

NAME	ROLES
GOKULNAATH M	DATA ENGINEER
JAYARAKSHA REGURAJ	STORYTELLER
KAMALESH N	DATA ANALYST
NISHANTH P	DATA VISUALIZATION

Work done:**1. Domain Selection and Problem Statement Abstract****Team Contribution:****1. Dataset Selection & Evaluation****Gokulnaath M – 24BAD028****Dataset 1****Heart Attack Prediction Dataset**

<https://www.kaggle.com/datasets/iamsouravbanerjee/heart-attack-prediction-dataset>

Characteristics

- Low / limited attributes
- Low object (record) count
- Semi-structured data

Pros

- Uncleaned dataset – ideal for demonstrating data preprocessing, handling missing values, and noise reduction
- Simple structure, easy to understand feature relationships
- Good for baseline model development

Cons

- Limited number of attributes restricts deep risk factor analysis
- Small dataset size reduces model generalization
- Not suitable alone for dimensionality reduction showcase

Jayaraksha Reguraj – 24BAD044**Dataset 2****Heart Disease Dataset**

<https://www.kaggle.com/datasets/johnsmith88/heart-disease-dataset>

Characteristics

- Already cleaned and pre-processed
- Low / limited object count

Pros

- Clean and ready for quick model training
- Good for algorithm comparison
- Minimal preprocessing effort required

Cons

- Already cleaned – not ideal for project requirements that demand full preprocessing pipeline
- Less opportunity to demonstrate data cleaning skills
- Limited data volume

Kamalesh N – 24BAD054**Dataset 3 (Finalized Dataset)****Heart Disease Prediction using Logistic Regression**

<https://www.kaggle.com/datasets/dileep070/heart-disease-prediction-using-logistic-regression>

Characteristics

- Good number of attributes
- Dataset is not pre-processed

Pros

- Uncleaned dataset – perfectly aligned with project requirement to perform:
 - Data cleaning
 - Feature scaling
 - Dimensionality reduction (PCA)
- Balanced attribute set supports risk score generation
- Suitable for end-to-end ML pipeline

Cons

- Requires significant preprocessing effort
- Noise and missing values may affect initial model performance
- Needs careful feature engineering

Nishanth P – 24BAD405

Dataset 4

Framingham Heart Study Dataset

<https://www.kaggle.com/datasets/amanajmera1/framingham-heart-study-dataset>

Characteristics

- Long-term cardiovascular study data
- Moderate to high attribute count
- Includes demographic, behavioural, and medical risk factors

Pros

- Raw / uncleaned dataset – excellent for showcasing:
 - Missing value handling
 - Outlier detection
 - Feature transformation
- Rich clinical depth enables strong risk factor interpretation
- Ideal for dimensionality reduction & risk score modeling

Cons

- Requires extensive preprocessing
- Some attributes may need domain understanding
- Higher complexity compared to smaller datasets

Summary Comparison Table

Dataset	Pre-processed	Attribute Count	Object Count	Suitability
Dataset 1	No	Low	Low	Basic preprocessing demo
Dataset 2	Yes	Low	Low	Model testing only
Dataset 3 (Final)	No	High	Medium	Best fit for project
Dataset 4	No	High	Medium	Advanced analysis

2. Setting up GitHub

All *

Procedure

- Create GitHub Repositories with name “[24ADI204 DSV Team14](#)”
- Add Description of
"Heartbeats & Habits" combats the silent threat of cardiovascular disease by applying rigorous cleaning and dimensionality reduction to noisy clinical data. This approach transforms complex biomarker and lifestyle interactions into a unified "Risk Score," visualizing a clear path to improved cardiac health

- Add README.md

3. Introduction to Pandas/NumPy

All *

Procedure

- Research and learn the basic concept of NumPy
- Research and learn the basic concept of Pandas

Resources and References

- **Dataset**
 - <https://www.kaggle.com/datasets/iamsouravbanerjee/heart-attack-prediction-dataset>
 - <https://www.kaggle.com/datasets/johnsmith88/heart-disease-dataset>
 - <https://www.kaggle.com/datasets/dileep070/heart-disease-prediction-using-logistic-regression>
 - <https://www.kaggle.com/datasets/amanajmera1/framingham-heart-study-dataset>
- **GitHub**
 - https://github.com/Gokulnaath-gif/24ADI204_DSV_Team14
- **Learns**
 - <https://numpy.org/doc/stable/user/index.html#user>
 - https://www.w3schools.com/python/numpy/numpy_intro.asp
 - https://pandas.pydata.org/docs/user_guide/index.html
 - <https://www.w3schools.com/python/pandas/default.asp>

Week No: 2**Know Your Data****Objectives:**

Loading data, checking types, inspecting the structure.

Work done:**1. Load Data and Analyses Dataset**

Gokulnaath M – 24BAD028, Nishanth P – 24BAD405

Procedure**1. Load the Dataset**

```
✓ import numpy as np  
import pandas as pd  
df=pd.read_csv("./data/framingham.csv")
```

- Libraries and imported as requirement
- Load the dataset from csv

2. Data Structure

```
df.info()
df.describe()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 4238 entries, 0 to 4237
Data columns (total 16 columns):
#   Column              Non-Null Count  Dtype
---  -
0   male                 4238 non-null   int64
1   age                  4238 non-null   int64
2   education            4133 non-null   float64
3   currentSmoker        4238 non-null   int64
4   cigsPerDay           4209 non-null   float64
5   BPMeds               4185 non-null   float64
6   prevalentStroke       4238 non-null   int64
7   prevalentHyp         4238 non-null   int64
8   diabetes             4238 non-null   int64
9   totChol              4188 non-null   float64
10  sysBP                4238 non-null   float64
11  diaBP                4238 non-null   float64
12  BMI                  4219 non-null   float64
13  heartRate            4237 non-null   float64
14  glucose              3850 non-null   float64
15  TenYearCHD           4238 non-null   int64
dtypes: float64(9), int64(7)
memory usage: 529.9 KB
```

- Get the basic information of dataset
- Columns non-null and data type

3. Data Statistical Information

	male	age	education	currentSmoker	cigsPerDay	BPMeds	prevalentStroke	prevalentHyp	diabetes	totChol	sysBP	diaBP	BMI	heartRate	glucose	TenYearCHD
count	4238.000000	4238.000000	4133.000000	4238.000000	4209.000000	4185.000000	4238.000000	4238.000000	4238.000000	4188.000000	4238.000000	4238.000000	4219.000000	4237.000000	3850.000000	4238.000000
mean	0.429212	49.584946	1.978950	0.494101	9.003089	0.029630	0.005899	0.310524	0.025720	236.721585	132.352407	82.893464	25.802008	75.878924	81.966753	0.151958
std	0.495022	8.572160	1.019791	0.500024	11.920094	0.169584	0.076587	0.462763	0.158316	44.590334	22.038097	11.910850	4.080111	12.026596	23.959998	0.359023
min	0.000000	32.000000	1.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	107.000000	83.500000	48.000000	15.400000	44.000000	40.000000	0.000000
25%	0.000000	42.000000	1.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	206.000000	117.000000	75.000000	23.070000	68.000000	71.000000	0.000000
50%	0.000000	49.000000	2.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	234.000000	128.000000	82.000000	25.400000	75.000000	78.000000	0.000000
75%	1.000000	56.000000	3.000000	1.000000	20.000000	0.000000	0.000000	1.000000	0.000000	263.000000	144.000000	89.875000	28.040000	83.000000	87.000000	0.000000
max	1.000000	70.000000	4.000000	1.000000	70.000000	1.000000	1.000000	1.000000	1.000000	696.000000	295.000000	142.500000	56.800000	143.000000	394.000000	1.000000

- The Basic statistical values for dataset
- Information means, min, max, std, etc

4. Data Structure and Inspecting Null Values

```
df.shape
```

```
(4238, 16)
```

```
df.isnull().sum()
```

```
male            0
age             0
education       105
currentSmoker   0
cigsPerDay      29
BPMeds          53
prevalentStroke 0
prevalentHyp    0
diabetes        0
totChol         50
sysBP           0
diaBP           0
BMI             19
heartRate       1
glucose         388
TenYearCHD      0
dtype: int64
```

- Its show the shape of dataset no (rows x column)
- Show count null values in each columns

2. “First Impression” report.

Jayaraksha Reguraj – 24BAD044, Kamalesh N – 24BAD054

Report

1. Dataset Overview

The dataset used for this analysis is the Framingham Heart Study dataset, which contains medical and lifestyle information of patients. The primary objective of the dataset is to predict the 10-year risk of coronary heart disease (CHD).

- Number of records: 4,240
- Number of features: 16
- Target variable: TenYearCHD (binary: 0 = No CHD, 1 = CHD)

2. Data Structure

- The dataset consists of numerical features (both integer and float types).
- Variables include:
 - Demographic features: age, sex
 - Behavioural features: smoking status, cigarettes per day
 - Medical history: blood pressure medication, diabetes, stroke history
 - Clinical measurements: cholesterol, systolic & diastolic BP, BMI, heart rate, glucose

3. Data Types

- Most features are of type int64 or float64.
- The target variable TenYearCHD is an integer categorical variable representing a binary outcome.
- No categorical string variables are present, reducing the need for encoding at this stage.

4. Missing Values

- Several columns contain missing values, including:
 - education
 - cigsPerDay
 - BPMeds
 - totChol
 - BMI
 - heartRate
 - glucose

5. Initial Observations

- The dataset appears realistic and noisy, which is expected in medical data.
- Feature scales vary significantly (e.g., age vs. cholesterol vs. glucose), suggesting the need for feature scaling before modeling.

- The target variable is imbalanced, with fewer positive CHD cases compared to negative ones, which may affect model performance.

6. Data Quality Assessment

- No duplicate rows were observed.
- No obvious data corruption or invalid values were detected during initial inspection.
- Presence of missing values is the main data quality concern.

7. Overall, First Impression

The dataset is:

- Well-structured
- Relevant for classification tasks
- Contains missing values
- Requires preprocessing and scaling
- Potential class imbalance in the target variable

Resources and References

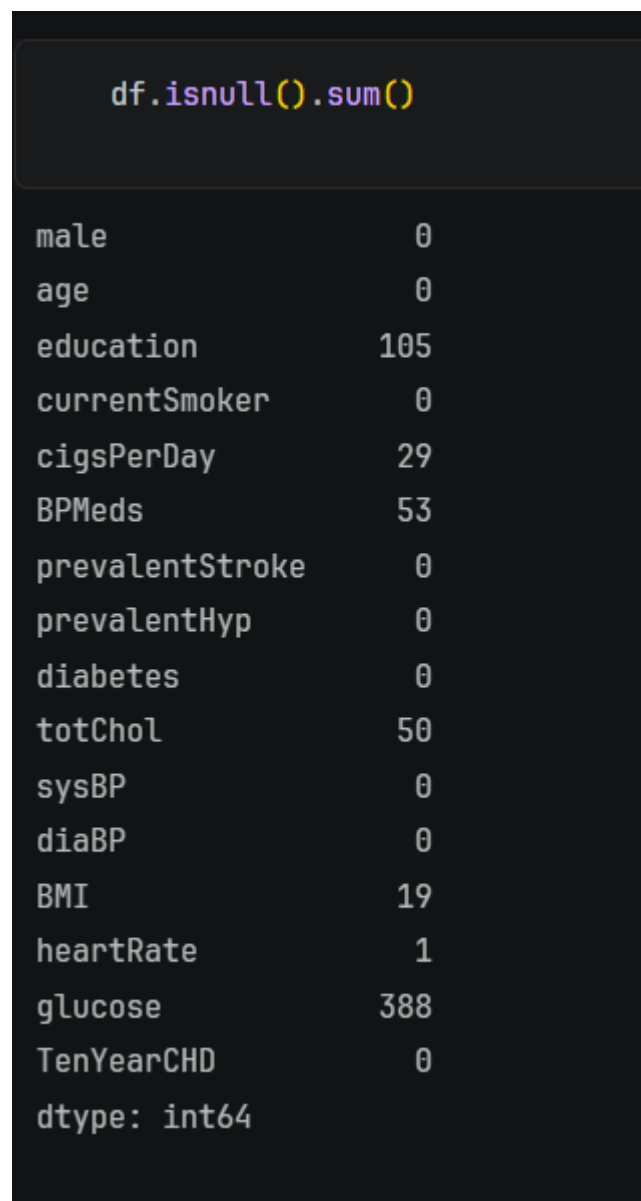
- **Dataset**
 - <https://www.kaggle.com/datasets/dileep070/heart-disease-prediction-using-logistic-regression>
- **GitHub**
 - https://github.com/Gokulnaath-gif/24ADI204_DSV_Team14

Week No: 3**The Cleaning Sprint****Objectives:**

Handling Missing Values (Imputation strategies) and Outlier Detection (Boxplots/Z-score).

Work done:**1. Handling Null Values**

Gokulnaath M – 24BAD028, Nishanth P – 24BAD405

1. Attributes Contains Null Values

```
df.isnull().sum()

male            0
age             0
education       105
currentSmoker   0
cigsPerDay      29
BPMeds          53
prevalentStroke 0
prevalentHyp    0
diabetes        0
totChol         50
sysBP           0
diaBP           0
BMI             19
heartRate       1
glucose         388
TenYearCHD      0
dtype: int64
```

- Null Values in columns

2. Selecting Categorical and Numerical Attributes

Selecting Numerical and Categorical Columns

- Categorical columns

```
categorical_cols = [  
    "education",  
    "currentSmoker",  
    "BPMeds",  
    "prevalentStroke",  
    "prevalentHyp",  
    "diabetes"  
]
```

- Numerical columns

```
numerical_cols = [  
    "cigsPerDay",  
    "totChol",  
    "BMI",  
    "heartRate",  
    "glucose",  
    "sysBP",  
    "diaBP"  
]
```

- Select the numerical columns and categorical columns

3. Imputation of Categorical Attributes (Mode)

Imputation: Categorical Attributes (Mode)

```
cat_imputer = SimpleImputer(strategy="most_frequent")  
df[categorical_cols] = cat_imputer.fit_transform(df[categorical_cols])
```

- Fill Null value of categorical datatype using mode

```
df.isnull().sum()
```

male	0
age	0
education	0
currentSmoker	0
cigsPerDay	29
BPMeds	0
prevalentStroke	0
prevalentHyp	0
diabetes	0
totChol	50
sysBP	0
diaBP	0
BMI	19
heartRate	1
glucose	388
TenYearCHD	0
dtype: int64	

- Now the Categorical column null values are fixed

4. Imputation of Numerical Attributes (Median)

Imputation: Numerical Attributes (Median)

```
num_imputer = SimpleImputer(strategy="median")

df[numerical_cols] = num_imputer.fit_transform(df[numerical_cols])
```

- Fill Null Values of Numerical Categories using median
- Why median because in medical records there some few high values so we don't use mean

output: Categorical Attributed Imputed

```
df.isnull().sum()
```

male	0
age	0
education	0
currentSmoker	0
cigsPerDay	0
BPMeds	0
prevalentStroke	0
prevalentHyp	0
diabetes	0
totChol	0
sysBP	0
diaBP	0
BMI	0
heartRate	0
glucose	0
TenYearCHD	0
dtype: int64	

- Now numerical null values are fixed

2. Handling Outliers values

Jayaraksha Reguraj – 24BAD044, Kamalesh N – 24BAD054

1. Outlier Detections using Boxplot

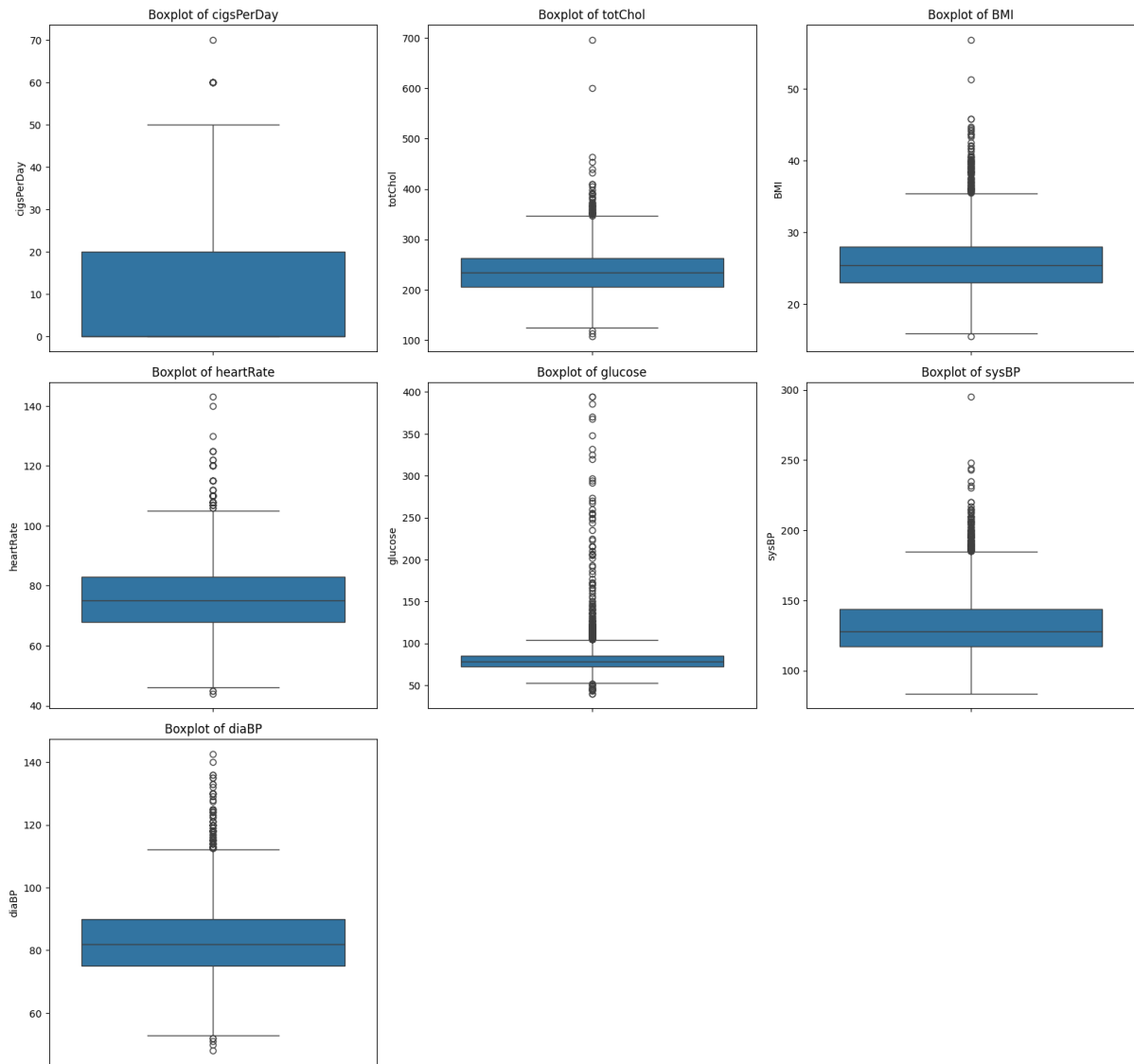
Outlier Dectection: Numerical attributes

- Using Boxplot

```
plt.figure(figsize=(16, 20))

for i, col in enumerate(numerical_cols, 1):
    plt.subplot(4, 3, i)
    sns.boxplot(y=df[col])
    plt.title(f"Boxplot of {col}")

plt.tight_layout()
plt.show()
```



- The output interpret that what are the numerical values
- From box plot attributes such glucose, diaBP, sysBP, BMI are highly containing outliers.

2. Outlier Detections using Z-Score

```
z_scores = df[numerical_cols].apply(zscore)

outliers = (np.abs(z_scores) > 3)

outlier_count = outliers.sum()
outlier_count
```

```
cigsPerDay    21
totChol       25
BMI           60
heartRate     23
glucose       50
sysBP         48
diaBP         41
dtype: int64
```

- Using Z-Score detect number of instances are outlier in each column
- From count of attributes such BMI, Glucose, sysBP, diaBP has contains outlier intense

3. Outlier Fixing

- Select Outlier Columns

```
outlier_cols = [  
    "cigsPerDay",  
    "totChol",  
    "BMI",  
    "heartRate",  
    "glucose",  
    "sysBP",  
    "diaBP"  
]
```

- Select the outlier attributes
 - IQR Capping Function

- IQR Capping Function

```
def iqr_capping(df, column):  
    Q1 = df[column].quantile(0.25)  
    Q3 = df[column].quantile(0.75)  
    IQR = Q3 - Q1  
  
    lower_bound = Q1 - 1.5 * IQR  
    upper_bound = Q3 + 1.5 * IQR  
  
    df[column] = df[column].clip(lower_bound, upper_bound)
```

- It allows to calculate IQR's upper & lower bound for each column
- It clip the column/attributes value with lower and upper bound values calculated from IQR

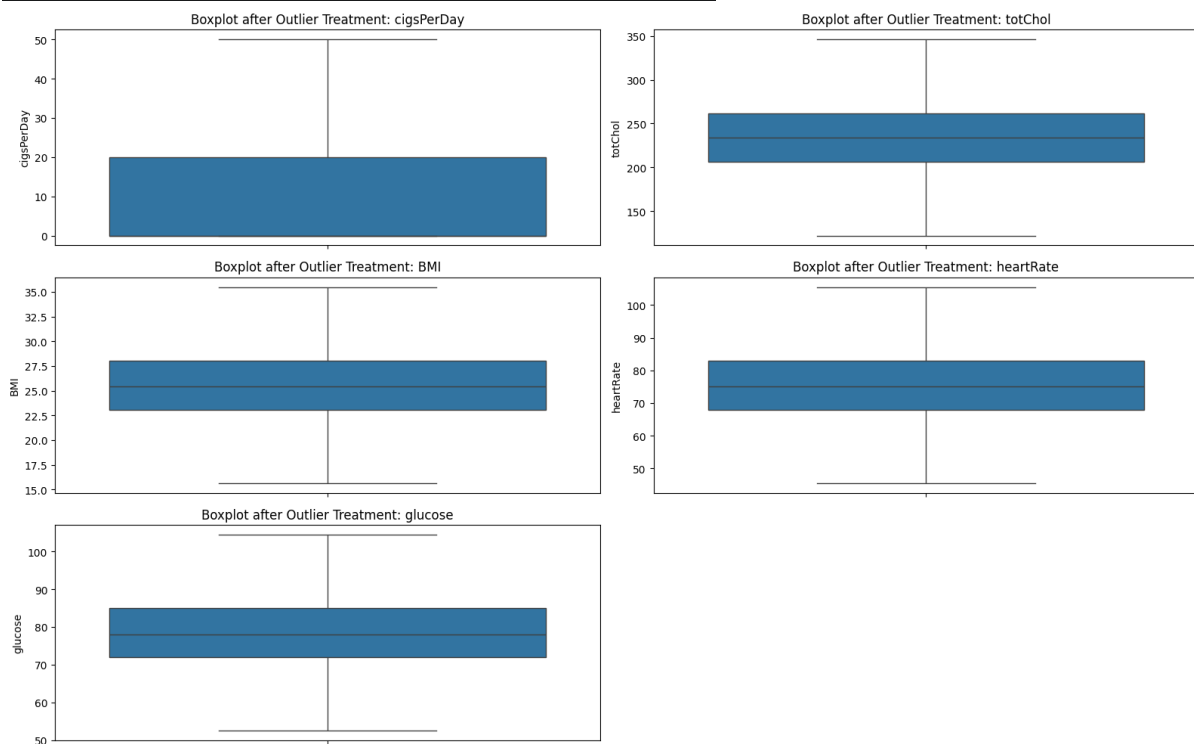
- Apply Capping to Each Attribute

```
for col in outlier_cols:  
    iqr_capping(df, col)
```

These loops iterate call IQR capping function for each column

- Verify Outliers After Treatment

```
plt.figure(figsize=(16, 20))  
  
for i, col in enumerate(outlier_cols, 1):  
    plt.subplot(4, 3, i)  
    sns.boxplot(y=df[col])  
    plt.title(f"Boxplot after Outlier Treatment: {col}")  
  
plt.tight_layout()  
plt.show()
```



Resources and References

- **Dataset**

- <https://www.kaggle.com/datasets/dileep070/heart-disease-prediction-using-logistic-regression>

- **GitHub**

- https://github.com/Gokulnaath-gif/24ADI204_DSV_Team14

- **Resources**

- <https://www.geeksforgeeks.org/data-analysis/working-with-missing-data-in-pandas/>
 - https://medium.com/@punya8147_26846/dataframes-handling-missing-values-in-pandas-11f7702afaf7
 - <https://www.geeksforgeeks.org/data-science/detect-and-remove-the-outliers-using-python/>
 - <https://www.geeksforgeeks.org/pandas/handling-outliers-with-pandas/>
 - <https://llego.dev/posts/outlier-detection-handling-python-guide/>
 - <https://x.ai/grok>
 - <https://openai.com/chatgpt>

Week No: 4

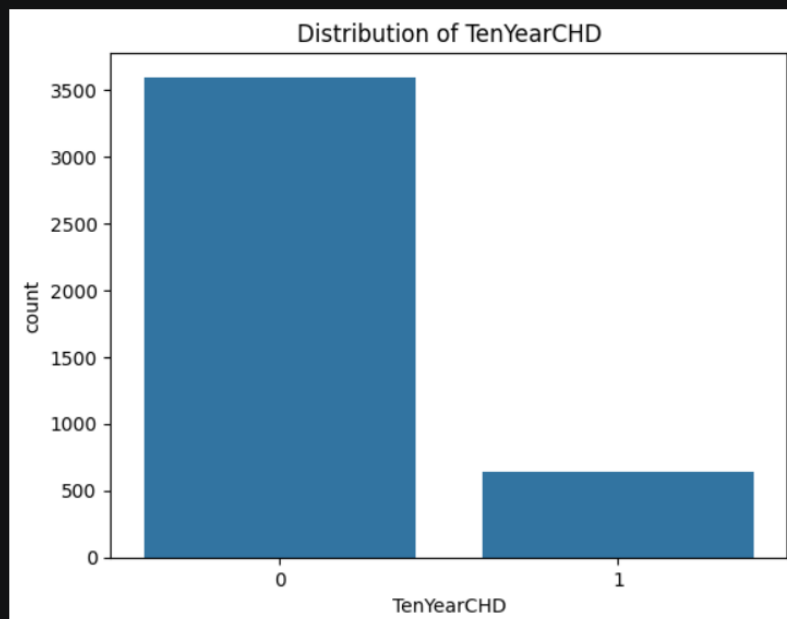
EDA Deep Dive

Objectives:

Variant analysis and Visualizing distributions.

Work done:**1. Variant Analysis****Kamalesh N 24BAD054 - Nishanth P 24BAD405****1. Univariate****1. TenYearCHD Distribution**

```
sns.countplot(x='TenYearCHD', data=df)  
plt.title("Distribution of TenYearCHD")  
plt.show()
```



- In your dataset, ~3700 records are not TenYearCHD and ~800 records are TenYearCHD

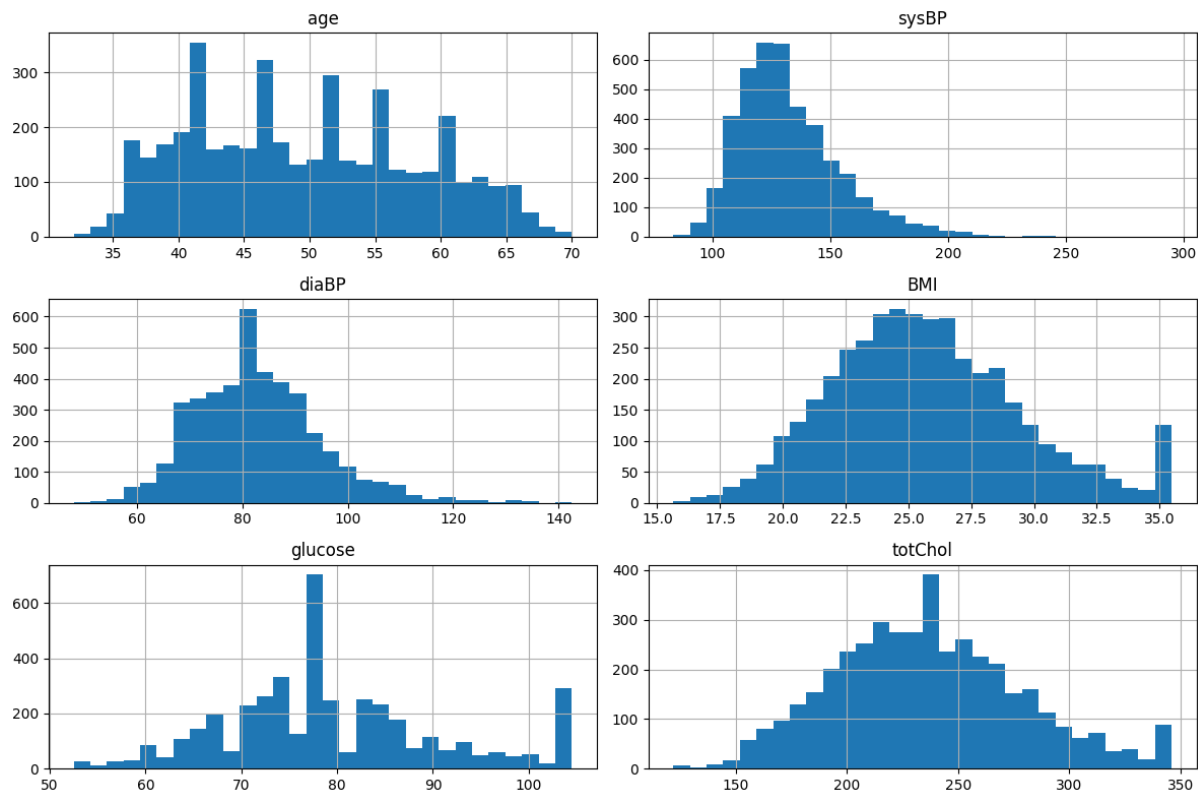
2. Continuous Variable Distribution

```
num_cols = ['age', 'sysBP', 'diaBP', 'BMI', 'glucose', 'totChol']

df[num_cols].hist(bins=30, figsize=(12,8))

plt.tight_layout()

plt.show()
```



1. **Most variables are roughly normal with skew:** age, diaBP, BMI, and totChol show near bell-shaped distributions, but with mild right skew (longer tail toward higher values).
2. **Strong right skew in sysBP and glucose:** sysBP and glucose have heavy right tails, indicating presence of high-value outliers (possible hypertensive or diabetic cases).
3. **BMI and cholesterol centered ranges:** BMI is concentrated around ~22–28, and total cholesterol around ~200–260, which are typical but include some high-risk values.
4. **Potential outliers present:** Extreme values (e.g., sysBP >200, glucose >100, totChol >320) suggest either true high-risk patients or data points that may need validation/preprocessing.

Why Histogram?

- Shows distribution shape (Normal / Skewed)
- Helps identify skewness

- Helps check extreme values

Univariate Observations

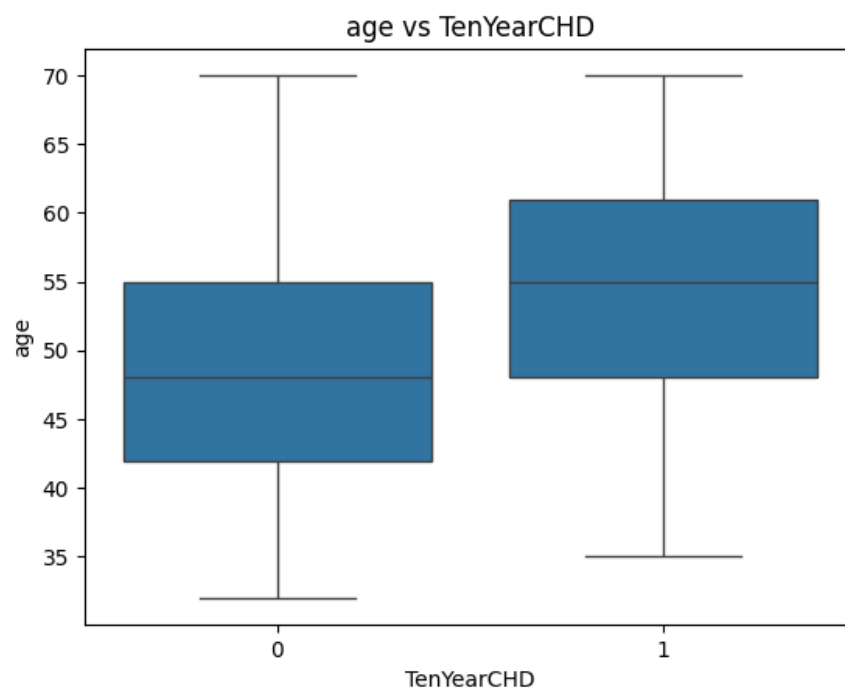
- age, BMI, glucose show right-skewed distributions
- sysBP and diaBP are approximately normal
- Outliers present in glucose and cholesterol

2. Bivariate

1. Numerical Feature vs Target

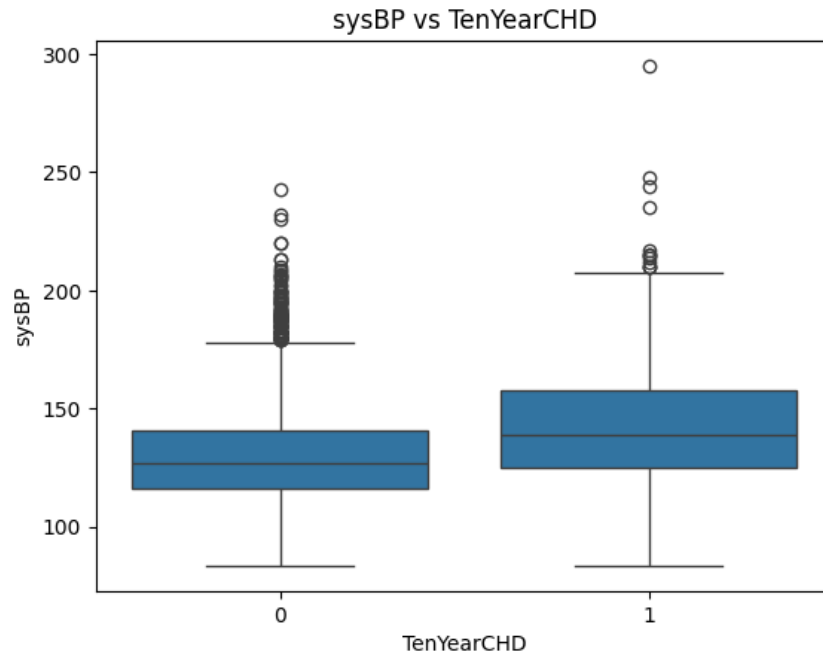
Numerical Feature vs Target

```
for col in ['age', 'sysBP', 'BMI', 'glucose']:  
    sns.boxplot(x='TenYearCHD', y=col, data=df)  
    plt.title(f"{col} vs TenYearCHD")  
    plt.show()
```

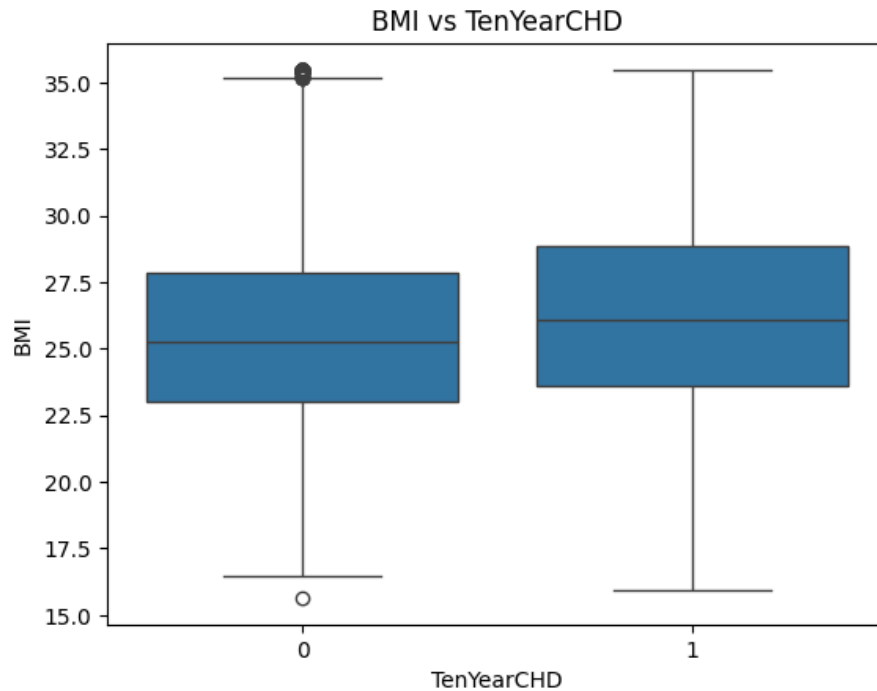


1. **Higher median age for CHD patients:** Individuals with TenYearCHD = 1 have a noticeably higher median age (~55) compared to those without CHD (~48), indicating age is positively associated with CHD risk.
2. **Shift in distribution:** The entire age distribution for CHD patients is shifted upward, suggesting that CHD cases are more concentrated in older age groups.

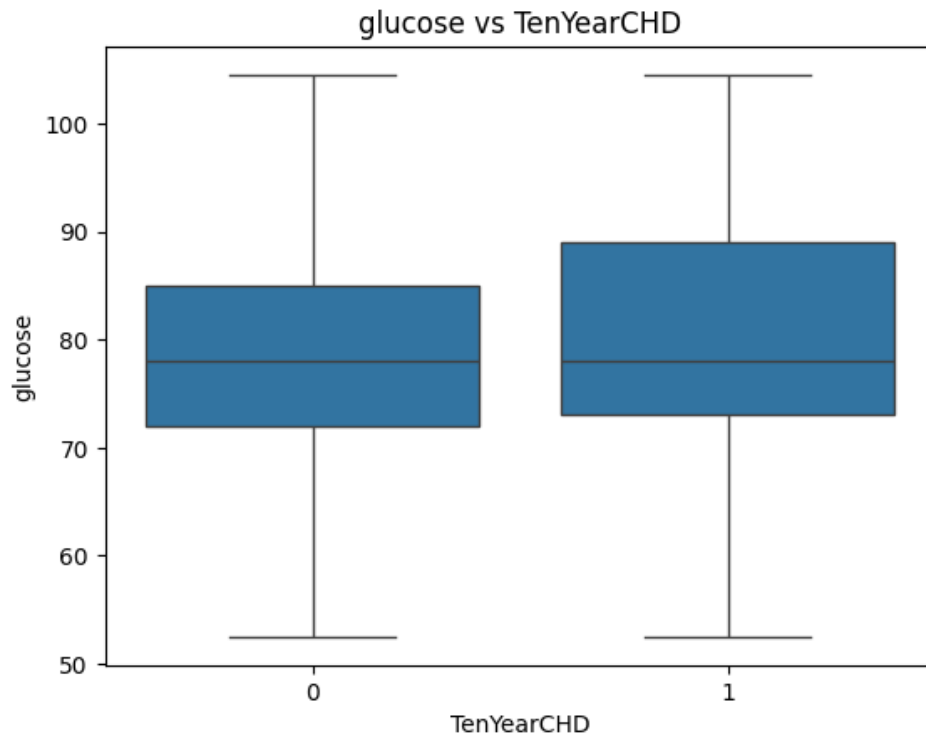
3. **Overlap exists:** There is still significant overlap between the two groups, meaning age alone is not sufficient to perfectly distinguish CHD vs non-CHD.
4. **Wider spread in CHD group:** The CHD group shows a slightly broader interquartile range, implying more variability in ages among affected individuals.



1. **Higher median sysBP in CHD group:** Individuals with TenYearCHD = 1 have a higher median systolic BP (~140) compared to non-CHD (~125), indicating a positive association.
2. **Upward shift in distribution:** The entire sysBP distribution for CHD patients is shifted higher, suggesting elevated blood pressure is a risk factor.
3. **Greater variability in CHD group:** Wider IQR and more spread in the CHD group imply more variation in systolic BP among affected individuals.
4. **More extreme outliers:** CHD group shows higher extreme values (up to ~300), reinforcing that very high systolic BP is linked with increased CHD risk.



1. **Slightly higher BMI in CHD group:** Median BMI for TenYearCHD = 1 (~26) is only marginally higher than non-CHD (~25), indicating a weak positive association.
2. **Significant overlap:** The IQRs of both groups overlap heavily, so BMI alone is a poor discriminator for CHD risk.
3. **Similar spread:** Both groups show comparable variability, suggesting BMI distribution is not drastically different between them.
4. **Limited predictive power:** Compared to variables like age or sysBP, BMI appears to be a less influential feature for predicting CHD in this dataset.

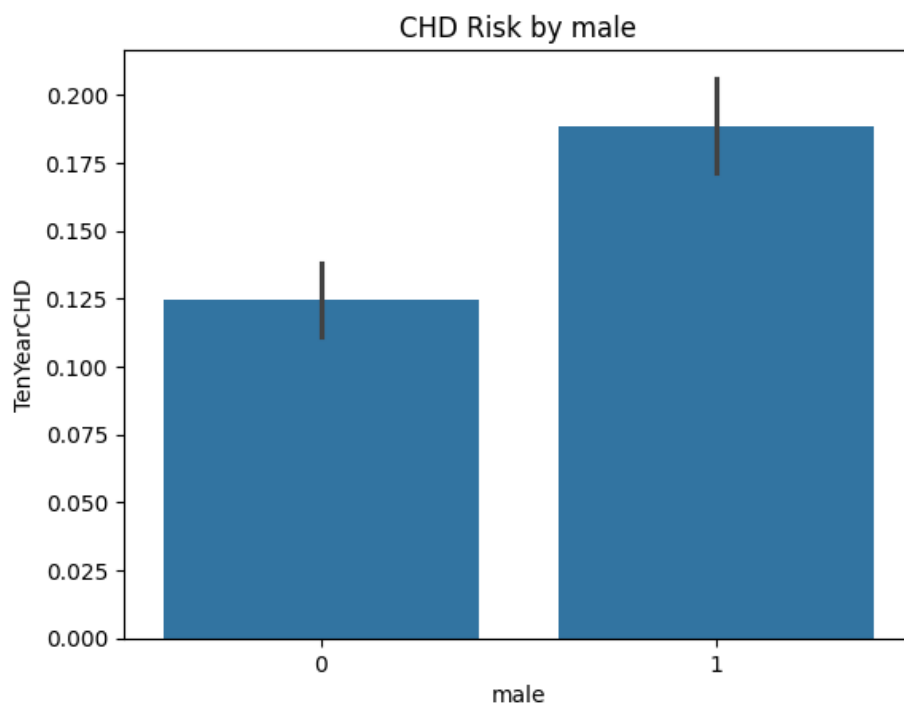


1. **Nearly identical medians:** Both groups (CHD = 0 and 1) have very similar median glucose ($\sim 77-78$), indicating weak separation.
2. **Heavy overlap in distributions:** The IQRs overlap significantly, so glucose alone does not clearly distinguish CHD vs non-CHD cases.
3. **Slight upward spread in CHD group:** The CHD group shows a marginally higher upper range, suggesting higher glucose may contribute but not strongly.
4. **Low standalone predictive power:** Compared to features like age or sysBP, glucose appears to have limited individual impact on CHD prediction.

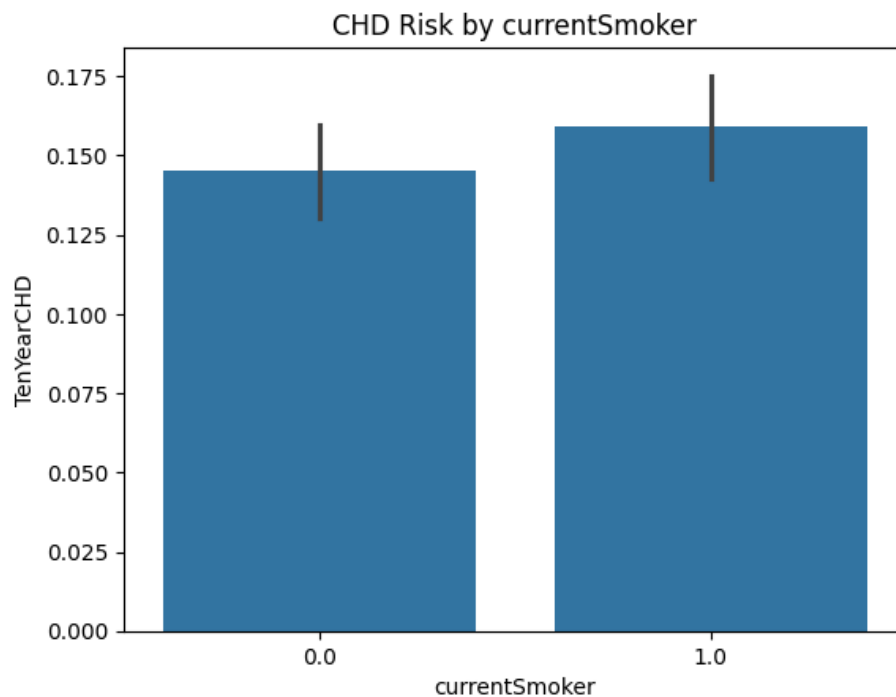
2. Categorical Features vs Target

```
cat_cols = ['male', 'currentSmoker', 'diabetes', 'prevalentHyp']

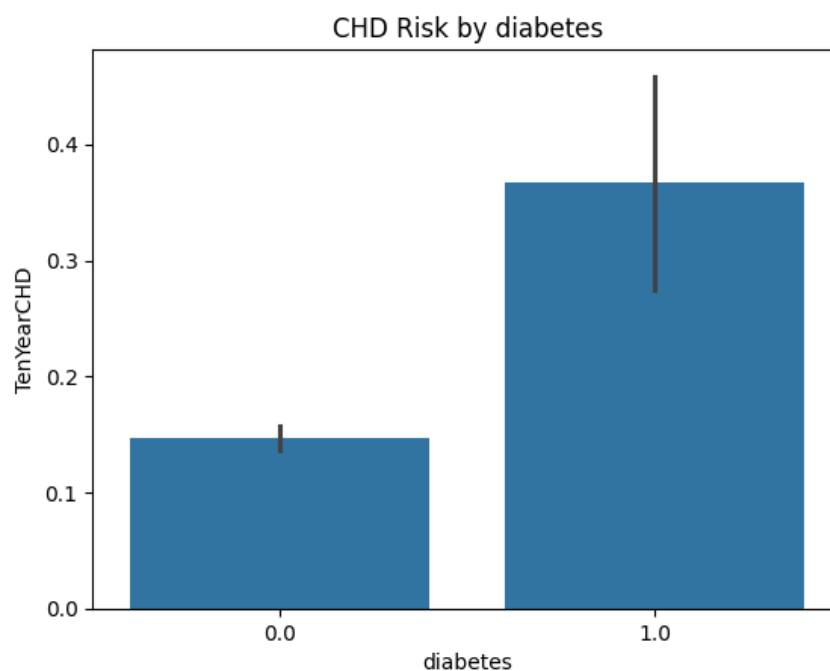
for col in cat_cols:
    sns.barplot(x=col, y='TenYearCHD', data=df)
    plt.title(f"CHD Risk by {col}")
    plt.show()
```



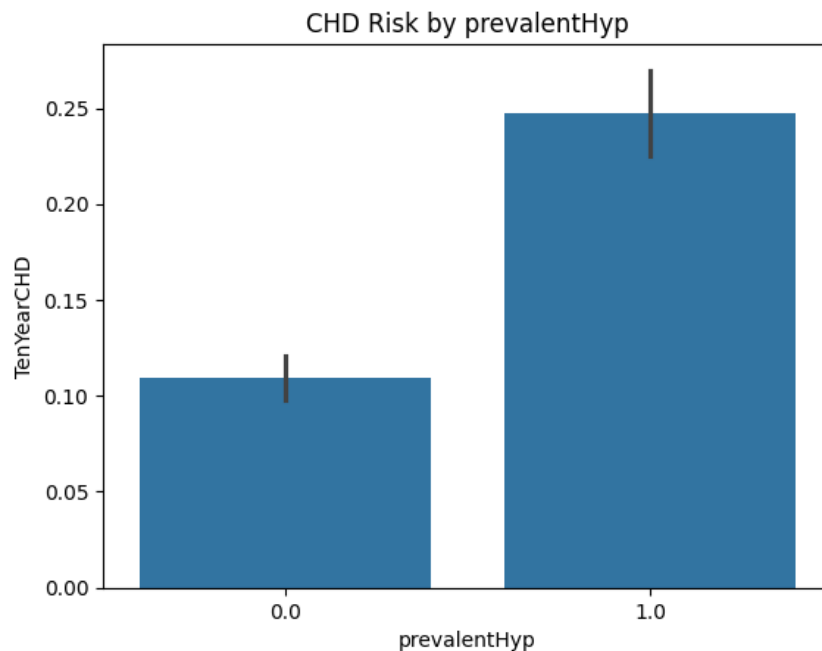
1. **Higher CHD risk in males:** Males (1) show a noticeably higher average TenYearCHD (~19%) compared to females (~12–13%).
2. **Clear group separation:** The difference between male and female CHD risk is substantial, indicating gender is an important predictive feature.
3. **Consistent trend (low overlap via error bars):** The error bars show limited overlap, suggesting the difference is statistically meaningful.
4. **Strong categorical predictor:** Unlike BMI or glucose, gender provides clearer discrimination for CHD risk in this dataset.



1. **Slightly higher CHD risk in smokers:** Current smokers (~16%) have a higher average CHD risk than non-smokers (~14–15%).
2. **Moderate difference:** The gap is present but not as large as with gender, indicating smoking contributes but is not the strongest factor alone.
3. **Overlapping error bars:** The confidence intervals overlap, suggesting the difference may not be strongly statistically significant.
4. **Contributory, not dominant feature:** Smoking likely interacts with other variables (e.g., age, BP) rather than acting as a strong standalone predictor.



1. **Much higher CHD risk in diabetics:** Individuals with diabetes (~37%) have significantly higher CHD risk compared to non-diabetics (~14–15%).
2. **Strong separation:** The difference is large and visually clear, indicating diabetes is a major risk factor.
3. **Higher variability in diabetics:** Wider error bars suggest more variation in CHD outcomes among diabetic patients.
4. **Highly predictive feature:** Compared to smoking or BMI, diabetes shows strong discriminative power for CHD risk.



1. **Significantly higher CHD risk with hypertension:** Individuals with prevalent hypertension (~25%) have much higher CHD risk than those without (~11%).
2. **Clear separation between groups:** The gap is large, indicating hypertension is a strong contributing factor.
3. **Moderate variability in hypertensive group:** Slightly wider error bars suggest some variation, but the trend remains consistent.
4. **Strong predictive feature:** Prevalent hypertension is one of the most influential variables for CHD risk, comparable to diabetes and stronger than smoking or BMI.

Bivariate Observations

- CHD patients have higher median age and blood pressure
- Diabetes and hypertension show strong association with CHD
- Smoking increases CHD probability

4. Multivariate Analysis

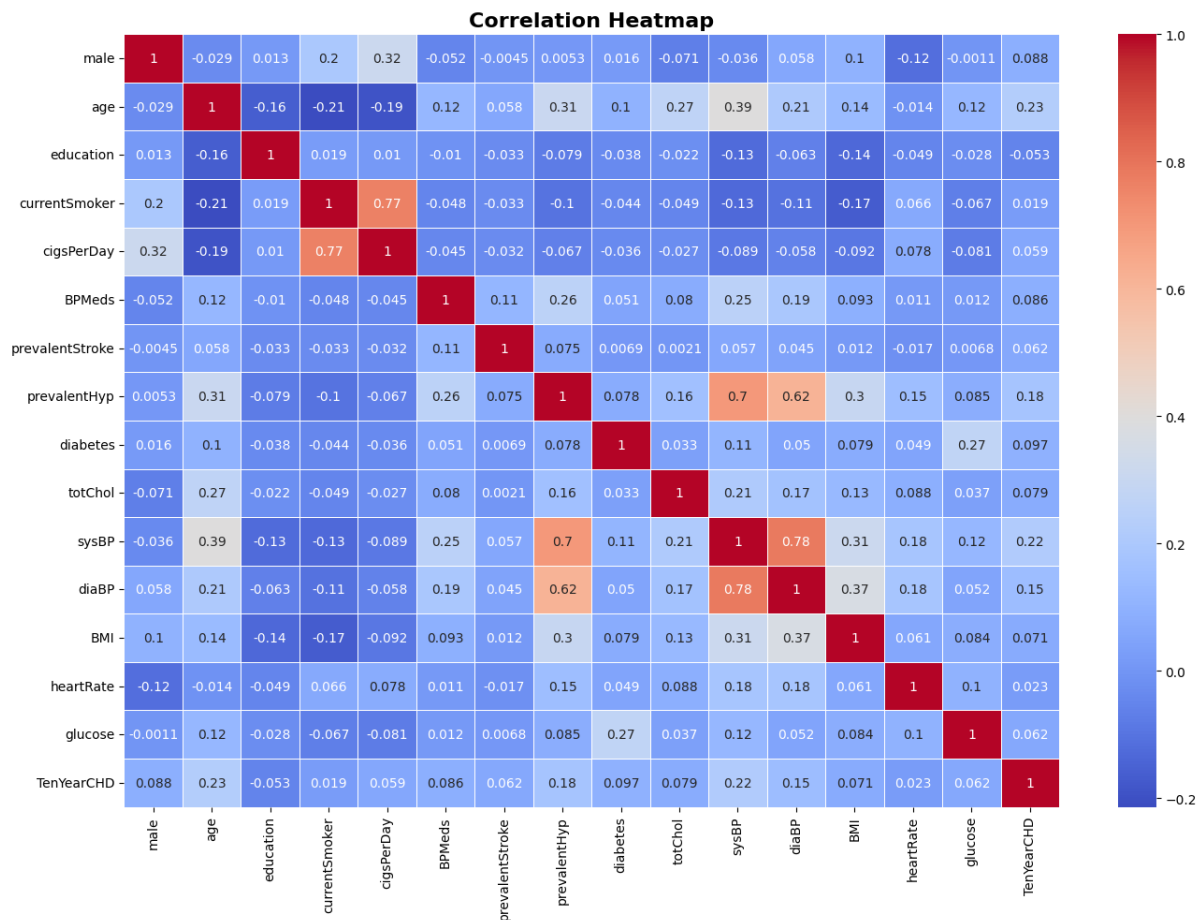
1. Correlation Matrix

```

mainjupyter M X
Generate + Code + Markdown | Interrupt Restart Clear All Outputs Go To | Jupyter Variables Outline ... Python 3.13.7

1 plt.figure(figsize=(14, 10))
2 sns.heatmap(
3     df.corr(),
4     mask=None,
5     cmap='coolwarm',
6     linewidths=0.5,
7     annot=True
8 )
9 plt.title("Correlation Heatmap", fontsize=16, fontweight='bold')
10 plt.tight_layout()
11 plt.show()
12
[53] Python

```



- Strong multicollinearity in BP features:** sysBP and diaBP show very high correlation (~ 0.78), and both are strongly correlated with prevalentHyp (~ 0.7 , ~ 0.62).
→ These features carry overlapping information.
- Smoking behavior linkage:** currentSmoker and cigsPerDay have very high correlation (~ 0.77), meaning one can largely represent the other in modeling.
- Key predictors for CHD (moderate correlation):**
 - age (~ 0.23)
 - sysBP (~ 0.22)
 - prevalentHyp (~ 0.18)
 - diaBP (~ 0.15)
 → These are the most relevant continuous predictors for TenYearCHD.

4. **Diabetes has moderate influence:** Correlation with CHD (~ 0.097) is weaker than expected but still positive—impact may be nonlinear or interaction-based.
5. **Weak predictors:** BMI, glucose, heartRate, and education show very low correlation with CHD → limited standalone predictive power.
6. **Age relationships:** Age correlates with several risk factors (sysBP, prevalentHyp, totChol), indicating it acts as a central driver variable.
7. **Low correlation $\neq$ no importance:** Some variables (like glucose, BMI) may still be important in nonlinear models (e.g., tree-based models), despite weak Pearson correlation.
8. **Feature redundancy warning:** Including highly correlated variables (e.g., sysBP + diaBP + prevalentHyp) in linear models can cause multicollinearity issues → requires feature selection or regularization.

2. Pair Plot

```
# Group 1: Demographic + lifestyle
cols1 = ['age', 'male', 'currentSmoker', 'cigsPerDay']

sns.pairplot(df[cols1 + ['TenYearCHD']], hue='TenYearCHD', diag_kind='kde', corner=True)
plt.suptitle("Demographic + Lifestyle Features", y=1.02, fontsize=16, fontweight='bold')
plt.show()

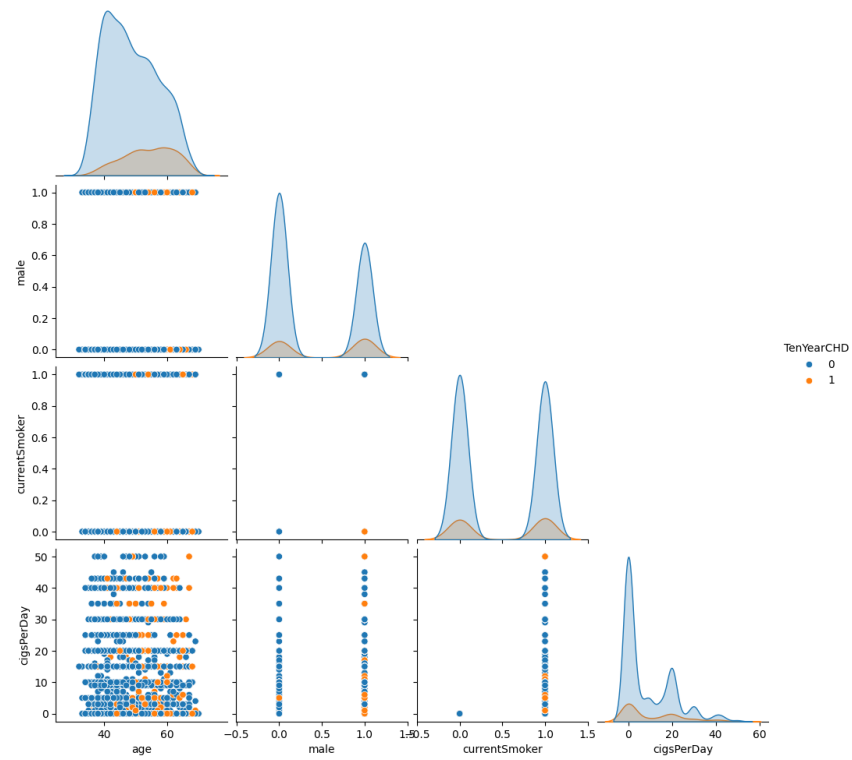
# Group 2: Medical history
cols2 = ['BPMeds', 'prevalentStroke', 'prevalentHyp', 'diabetes']

sns.pairplot(df[cols2 + ['TenYearCHD']], hue='TenYearCHD', diag_kind='kde', corner=True)
plt.suptitle("Medical History / Existing Conditions", y=1.02, fontsize=16, fontweight='bold')
plt.show()

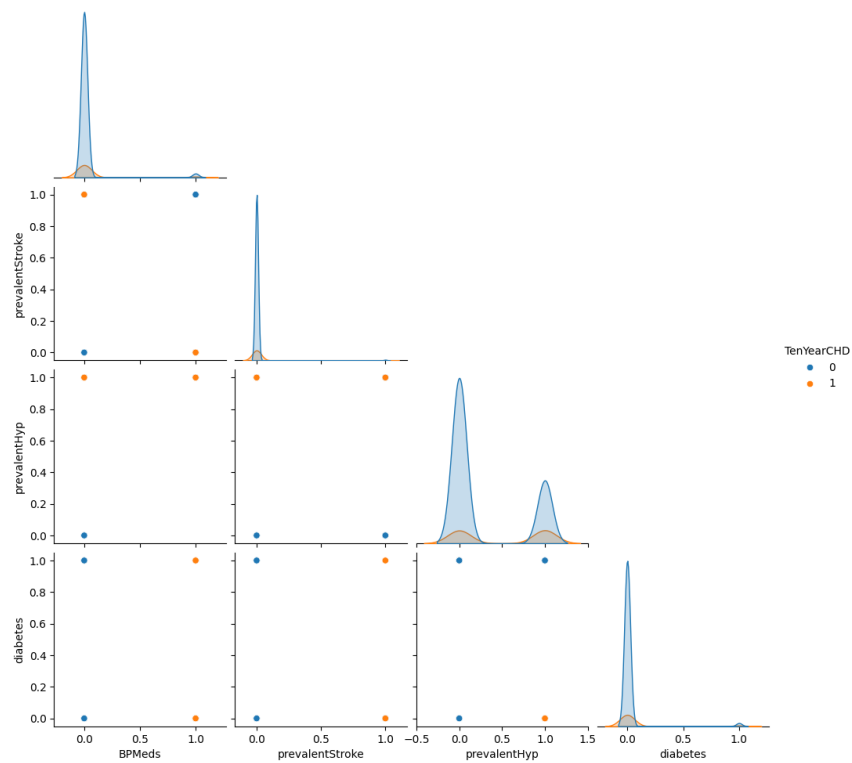
# Group 3: Clinical measurements
cols3 = ['totChol', 'sysBP', 'diaBP', 'BMI', 'heartRate', 'glucose']

sns.pairplot(df[cols3 + ['TenYearCHD']], hue='TenYearCHD', diag_kind='kde', corner=True)
plt.suptitle("Clinical / Measured Health Parameters", y=1.02, fontsize=16, fontweight='bold')
plt.show()
```

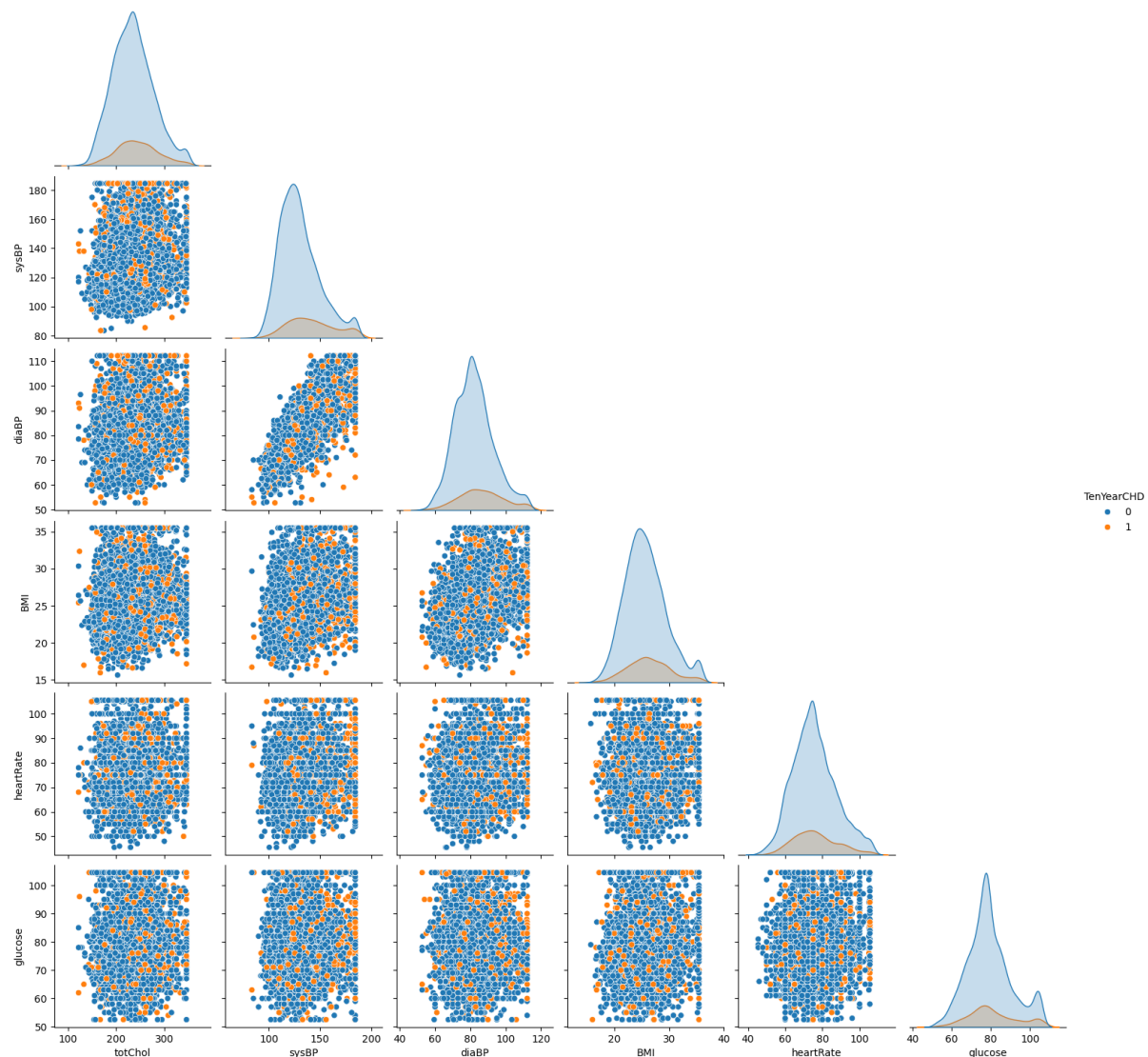
Demographic + Lifestyle Features



Medical History / Existing Conditions



Clinical / Measured Health Parameters



1. **Age shows clear separation trend:** In the demographic plot, CHD cases (orange) are more concentrated at higher ages → age remains one of the strongest drivers.
2. **Smoking intensity matters more than status:** `cigsPerDay` shows a wider spread for CHD cases compared to just `currentSmoker` → quantity is more informative than binary smoking status.
3. **Gender effect visible but not dominant in interactions:** While males have higher CHD risk, pairwise plots show overlap → gender alone doesn't create clear clusters.
4. **Hypertension strongly clusters with CHD:** In medical history plots, `prevalentHyp = 1` has visibly more CHD cases → strong categorical predictor.
5. **Diabetes shows sparse but high-risk pattern:** Few diabetic samples, but many are CHD-positive → high impact, low frequency feature.
6. **Blood pressure relationships are structured:** `sysBP` vs `diaBP` shows a clear linear trend → confirms strong correlation and potential redundancy.
7. **CHD cases shift toward higher BP values:** In scatter plots, CHD points are denser in higher `sysBP` and `diaBP` regions → reinforces BP as a key risk factor.

8. **BMI and heartRate show weak separation:** Heavy overlap between CHD and non-CHD → low discriminative power individually.
9. **Glucose has slight upward shift:** CHD cases appear marginally higher in glucose, but overlap is large → weak standalone predictor.
10. **No clear nonlinear clusters:** Data is highly overlapping across most feature pairs → suggests need for multivariate models (e.g., logistic regression, tree-based models) rather than relying on simple thresholds.

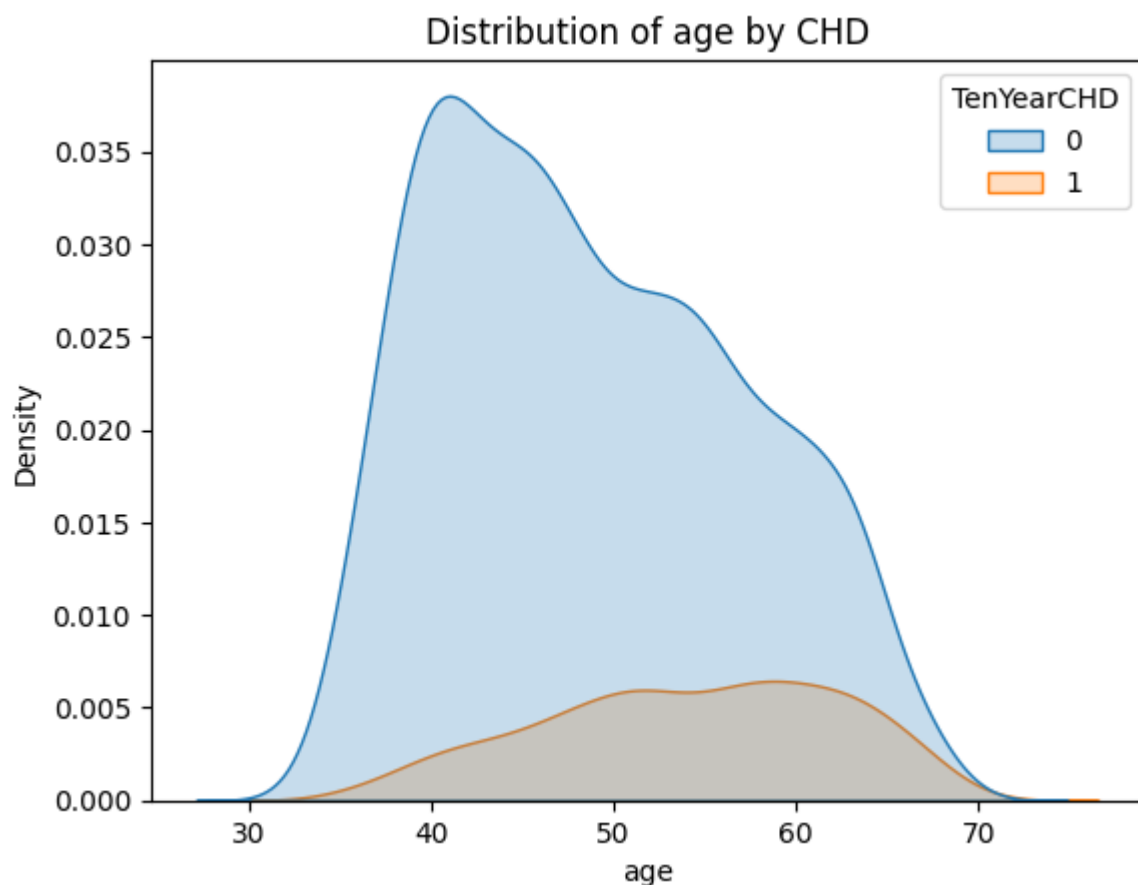
Multivariate Observations

- Strong correlation between sysBP and diaBP
- BMI and glucose are positively correlated
- Multicollinearity exists and affects linear models

2. Visualizing distributions

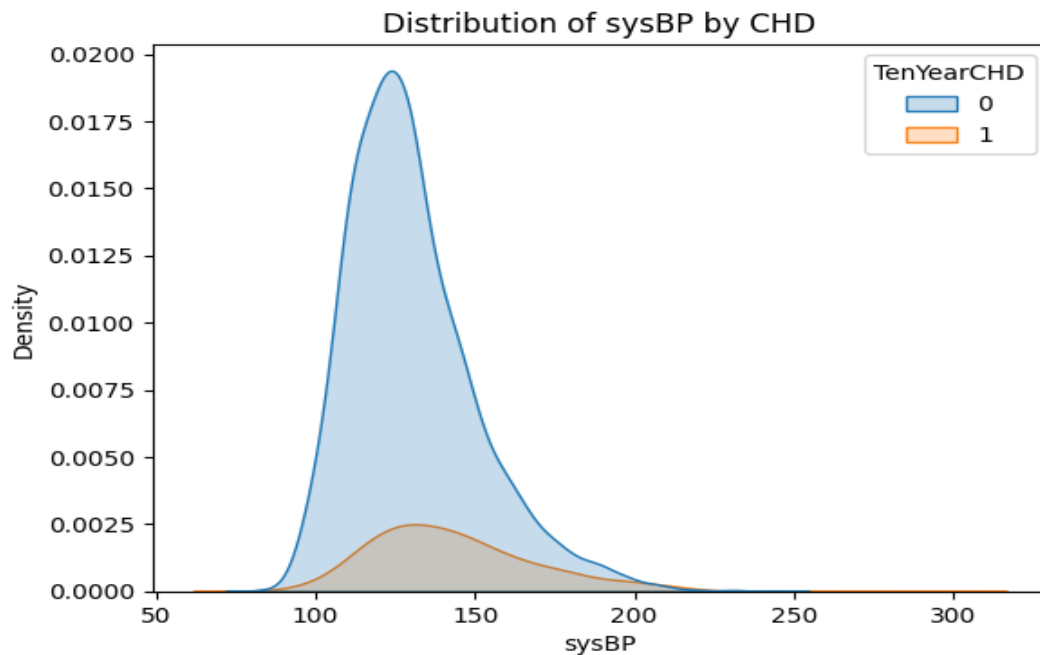
Gokulnaath M 24BAD028 - Jayaraksha Reguraj 24BAD044

1. KDE Distribution by Target

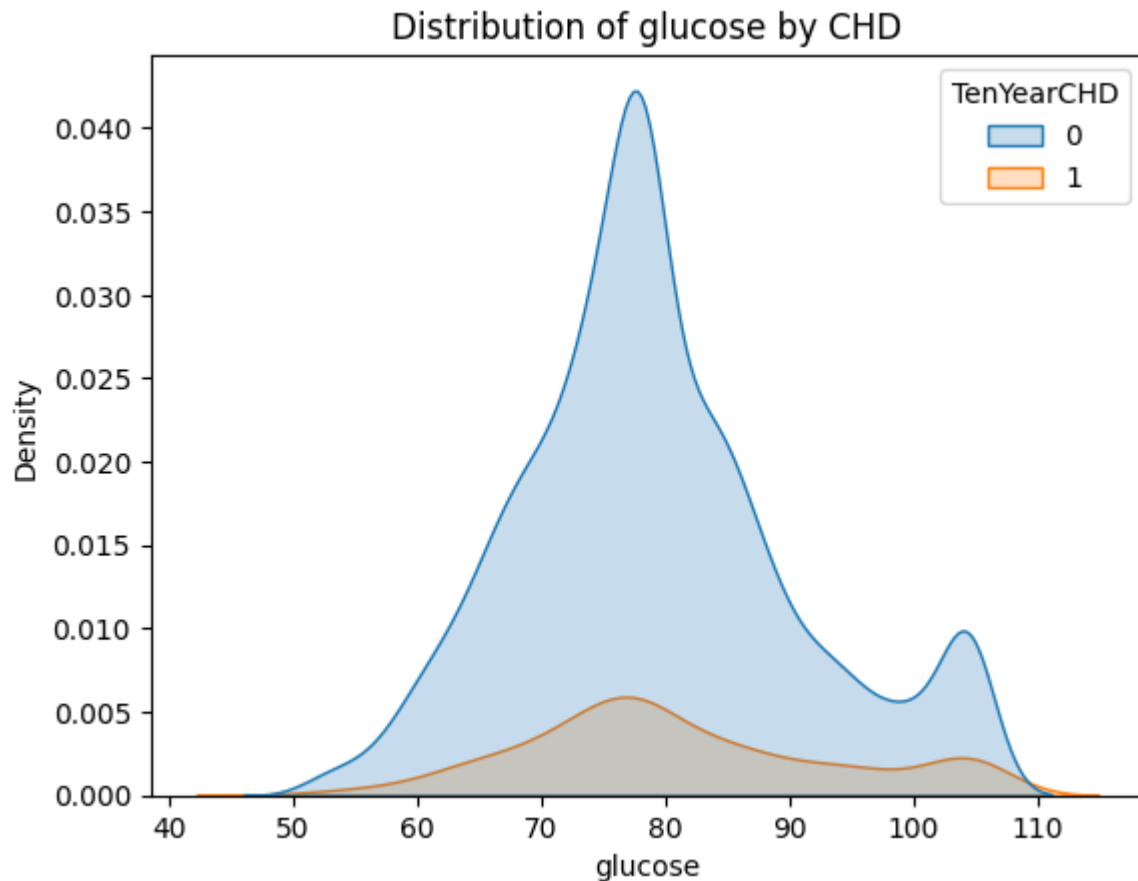


1. **CHD shifts toward older ages:** The CHD (1) distribution is clearly shifted right, with higher density in the ~50–65 range → strong age-risk relationship.

2. **Non-CHD concentrated in younger range:** The non-CHD group peaks around ~40–45, indicating lower risk at younger ages.
3. **Overlap still exists:** There is noticeable overlap (~45–60), so age alone cannot perfectly separate classes.
4. **Right-skewed CHD distribution:** CHD cases extend further into older ages (up to ~70+), reinforcing age as a progressive risk factor.



1. **Higher sysBP in CHD group:** The CHD (1) distribution is shifted toward higher systolic BP (~130–160), confirming strong association with elevated BP.
2. **Non-CHD concentrated at lower BP:** Most non-CHD cases peak around ~110–130, indicating lower risk in normal BP ranges.
3. **Long right tail in CHD:** CHD distribution extends further (200+), showing presence of extreme hypertension among affected individuals.
4. **Overlap exists but shifted:** There is overlap (~120–150), but overall density for CHD is consistently higher at elevated BP levels → useful predictive feature.



- **Very similar distributions:** Both CHD and non-CHD groups peak around $\sim 75\text{--}80 \rightarrow$ minimal shift between classes.
- **Heavy overlap:** The curves almost completely overlap, indicating poor separability using glucose alone.
- **Slight right tail in CHD:** CHD group shows a marginally higher density in the upper range ($\sim 90\text{--}110$), but the effect is weak.
- **Low predictive strength:** Glucose, as a standalone feature, has limited usefulness for distinguishing CHD risk in this dataset.

Resources and References

- **Dataset**

- <https://www.kaggle.com/dileep070/heart-disease-prediction-using-logistic-regression>

- **GitHub**

- https://github.com/Gokulnaath-gif/24ADI204_DSV_Team14

- **Resources**

- <https://www.geeksforgeeks.org/data-analysis/what-is-exploratory-data-analysis/>
 - <https://www.ibm.com/think/topics/exploratory-data-analysis>
 - <https://www.kaggle.com/code/imoore/intro-to-exploratory-data-analysis-eda-in-python>
 - <https://x.ai/grok>
 - <https://openai.com/chatgpt>
 - <https://google.com/gemini>

Week No: 6

Feature Engineering

Objectives:

To preprocess and prepare raw data for analysis and modeling by applying techniques such as normalization and scaling to ensure numerical features are on a comparable scale, encoding categorical variables to convert them into machine-readable formats, and fixing skewness in data distributions to improve model performance and statistical validity.

Work done:

1. Column Taxonomy

Gokulnaath M – 24BAD028

Group	Columns	Reason
Binary / Nominal	male, currentSmoker, BPMeds, prevalentStroke, prevalentHyp, diabetes	Already 0/1 — no encoding or scaling needed
Ordinal	education	4-level ordered category (1 < 2 < 3 < 4) → ordinal encoding
Continuous	age, cigsPerDay, totChol, sysBP, diaBP, BMI, heartRate, glucose	Require skew check + scaling
Target	TenYearCHD	Not transformed

```
binary_cols = ['male', 'currentSmoker', 'BPMeds', 'prevalentStroke', 'prevalentHyp', 'diabetes']
ordinal_cols = ['education']
continuous_cols = ['age', 'cigsPerDay', 'totChol', 'sysBP', 'diaBP', 'BMI', 'heartRate', 'glucose']
target_col = 'TenYearCHD'
```

- What Current Attributes at state before processing the skewness & categorical encoding & normalization

2. Skewness Detection

6.2 Skewness Detection

Skewness > 0.5 (moderate) or > 1.0 (high) indicates the distribution is asymmetric enough to distort distance-based models and slow gradient descent.

Range	Interpretation
-0.5 to 0.5	Approximately symmetric — no fix needed
0.5 to 1.0	Moderate right skew — apply transformation
> 1.0	High right skew — strong transformation required

```
skewness = df[continuous_cols].skew().sort_values(ascending=False)
print("Skewness of continuous features:\n")
print(skewness.to_string())
```

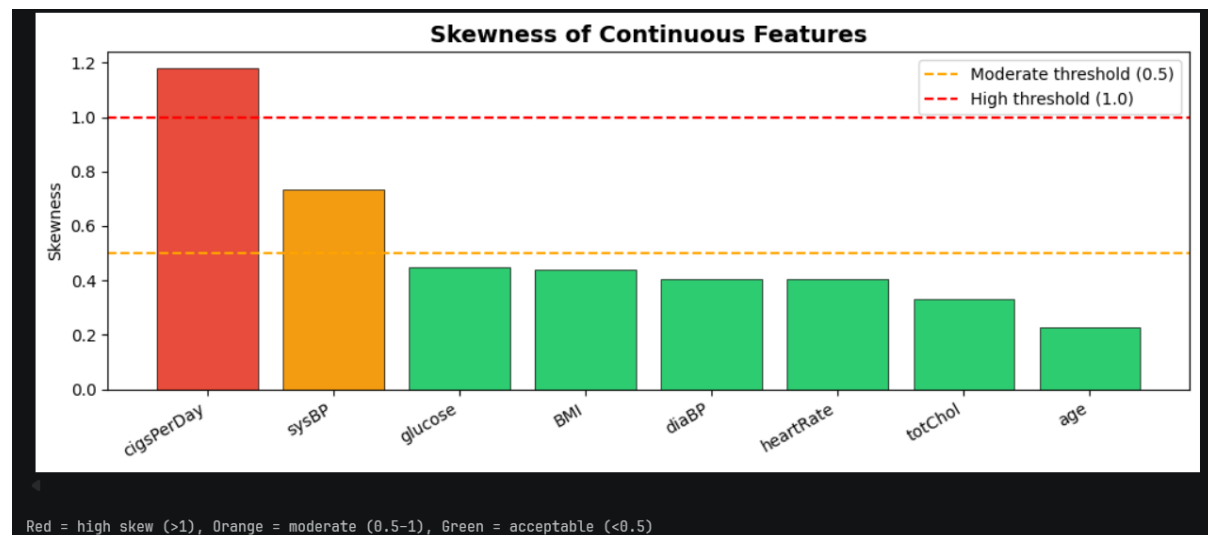
Skewness of continuous features:

```
cigsPerDay    1.188184
sysBP         0.732407
glucose       0.449353
BMI           0.438295
diaBP         0.404980
heartRate     0.403390
totChol       0.330328
age           0.228146
```

- This show skewness for each attribute calculated

Skewness chart

```
plt.figure(figsize=(10, 4))
colors = ['#e74c3c' if abs(s) > 1 else '#f39c12' if abs(s) > 0.5 else '#2ecc71'
         ]
for s in skewness.values]
bars = plt.bar(skewness.index, skewness.values, color=colors, edgecolor='black', linewidth=0.5)
plt.axhline(0.5, color='orange', linestyle='--', label='Moderate threshold (0.5)')
plt.axhline(1.0, color='red', linestyle='--', label='High threshold (1.0)')
plt.title("Skewness of Continuous Features", fontsize=14, fontweight='bold')
plt.ylabel("Skewness")
plt.xticks(rotation=30, ha='right')
plt.legend()
plt.tight_layout()
plt.show()
print("Red = high skew (>1), Orange = moderate (0.5-1), Green = acceptable (<0.5)")
```



1. **Highly skewed feature:** cigsPerDay (>1) is strongly right-skewed → many low/zero values with few heavy smokers. Needs transformation (e.g., log or binning).
2. **Moderately skewed feature:** sysBP (~0.7) shows noticeable right skew → presence of high BP outliers. Consider scaling or log transform.
3. **Near-normal features:** glucose, BMI, diaBP, and heartRate (~0.4–0.45) have acceptable skew → minimal preprocessing needed.
4. **Well-behaved features:** totChol (~0.33) and age (~0.22) are closest to symmetric → ideal for linear models.
5. **Modeling implication:** Skewed features can distort linear models → apply transformations or use tree-based models (which are robust to skewness).

Kamalesh N – 24BAD054

3. Fixing Skewness

6.3 Fixing Skewness — Log1p Transform on `cigsPerDay`

`cigsPerDay` has the highest skew (**1.18**) because most patients smoke 0 cigarettes (non-smokers) while a few smoke 40+. We apply **log1p** ($\log(1 + x)$) which:

- Handles zero values safely ($\log(0)$ is undefined; $\log1p(0) = 0$)
- Compresses the long right tail
- Reduces skew from 1.18 $\rightarrow$ ~ 0.32

We **replace** the original column so the pipeline is clean.

```
# Before
print(f"cigsPerDay skew BEFORE: {df['cigsPerDay'].skew():.4f}")

df['cigsPerDay'] = np.log1p(df['cigsPerDay'])

# After
print(f"cigsPerDay skew AFTER : {df['cigsPerDay'].skew():.4f}")

cigsPerDay skew BEFORE: 1.1802
cigsPerDay skew AFTER : 0.3200
```

1. **Handles zero values safely (critical advantage):** Unlike pure log or Box-Cox, $\log1p(x)$ works when $x = 0$ (common in `cigsPerDay` due to non-smokers). This avoids undefined values and keeps the dataset intact without imputation or shifting.
2. **Minimal distortion with effective skew reduction:** Log transform compresses extreme right-tail values while preserving relative ordering and interpretability better than aggressive methods (e.g., quantile transform), making it more suitable for linear models and statistical analysis.

Visual Comparison

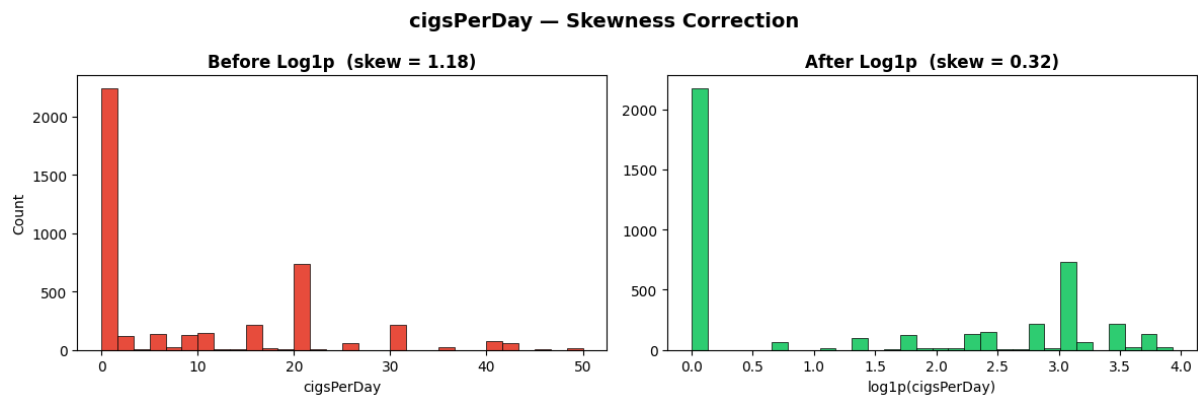
Visual Comparison — Before vs After Log Transform

```
fig, axes = plt.subplots(1, 2, figsize=(12, 4))

orig_values = pd.read_csv("./data/ab.csv", index_col=0)['cigsPerDay']
axes[0].hist(orig_values, bins=30, color='e74c3c', edgecolor='black', linewidth=0.5)
axes[0].set_title(f"Before Log1p (skew = {orig_values.skew():.2f})", fontweight='bold')
axes[0].set_xlabel("cigsPerDay")
axes[0].set_ylabel("Count")

axes[1].hist(df['cigsPerDay'], bins=30, color='2ecc71', edgecolor='black', linewidth=0.5)
axes[1].set_title(f"After Log1p (skew = {df['cigsPerDay'].skew():.2f})", fontweight='bold')
axes[1].set_xlabel("log1p(cigsPerDay)")

plt.suptitle("cigsPerDay - Skewness Correction", fontsize=14, fontweight='bold')
plt.tight_layout()
plt.show()
```



1. **Skewness significantly reduced:** The distribution shifts from highly right-skewed (1.18) to near-normal (~ 0.32), making it more suitable for statistical modeling.
2. **Outliers compressed effectively:** Large values (heavy smokers) are pulled closer to the main distribution, reducing their disproportionate influence.
3. **Improved model compatibility:** The transformed feature better satisfies assumptions of linear models (normality, homoscedasticity), leading to more stable and reliable predictions.

Nishanth P – 24BAD405

4. Ordinal Encoding (Education)

```
print("Before:", df['education'].dtype, "| Sample:", df['education'].unique())

df['education'] = df['education'].astype(int)

print("After :", df['education'].dtype, "| Sample:", sorted(df['education'].unique()))
```

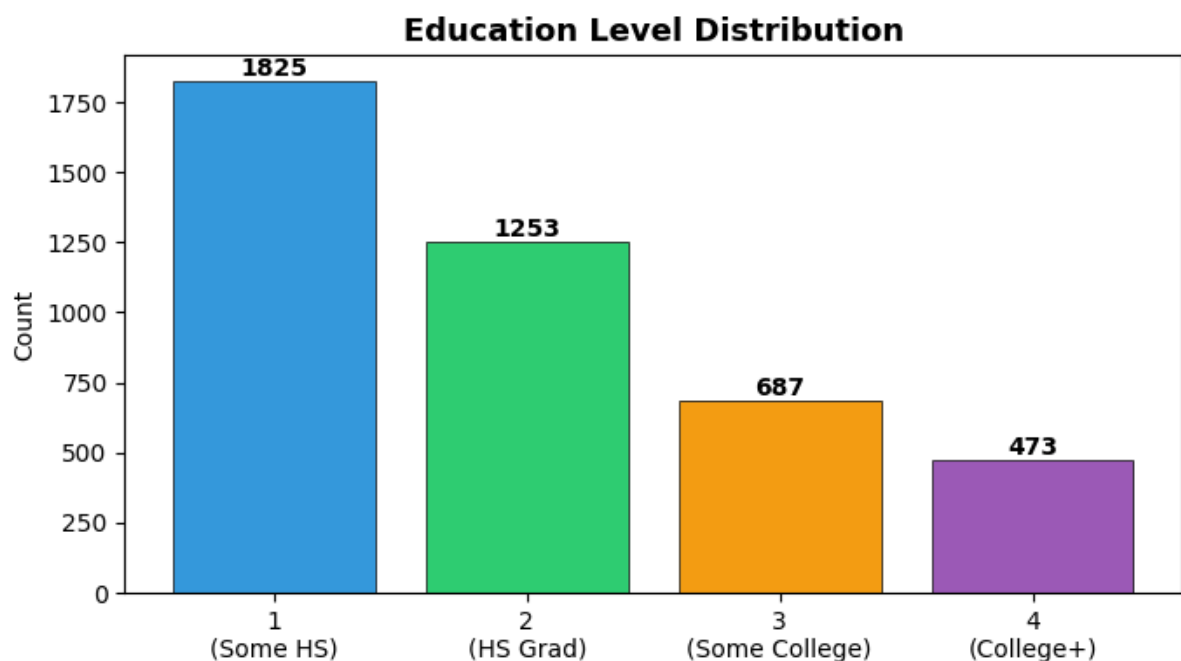
Before: float64 | Sample: [4. 2. 1. 3.]

After : int64 | Sample: [np.int64(1), np.int64(2), np.int64(3), np.int64(4)]

- Education levels have a natural order: 1 (Some HS) < 2 (HS Graduate) < 3 (Some College) < 4 (College), so ordinal encoding preserves meaningful ranking.
- One-Hot Encoding removes this order by splitting into separate binary columns, making the model unable to learn relative progression between levels.
- Therefore, ordinal encoding is preferred for ordered categories like education.

Visualization

```
plt.figure(figsize=(7, 4))
counts = df['education'].value_counts().sort_index()
bars = plt.bar(['1\n(Some HS)', '2\n(HS Grad)', '3\n(Some College)', '4\n(College+)'],
               counts.values, color=['#3498db', '#2ecc71', '#f39c12', '#9b59b6'],
               edgecolor='black', linewidth=0.5)
for bar, val in zip(bars, counts.values):
    plt.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 20,
             str(val), ha='center', fontweight='bold')
plt.title("Education Level Distribution", fontsize=13, fontweight='bold')
plt.ylabel("Count")
plt.tight_layout()
plt.show()
```



1. **Imbalanced distribution:** Majority of individuals are in lower education levels (1 and 2), with level 1 (Some HS) being the largest group → dataset is skewed toward lower education.
2. **Decreasing trend with higher education:** Counts consistently drop as education level increases → fewer individuals in higher education categories.
3. **Potential bias risk:** Models may learn patterns biased toward lower education groups due to higher representation → consider stratification or weighting if needed.

Scaling Continuous Features

- **StandardScaler** uses mean and standard deviation, so it is sensitive to outliers and works best when data follows a normal (Gaussian) distribution.
- **MinMaxScaler** rescales data using min and max values, making it highly sensitive to outliers and suitable only when there are no extreme values.
- **RobustScaler** uses median and IQR, making it resistant to outliers and the best choice when the dataset contains extreme values.

```
scale_cols = ['age', 'totChol', 'sysBP', 'diaBP', 'BMI', 'heartRate', 'glucose', 'cigsPerDay']

scaler = RobustScaler()
df[scale_cols] = scaler.fit_transform(df[scale_cols])

print("After RobustScaler - descriptive statistics:")
df[scale_cols].describe().round(3)
```

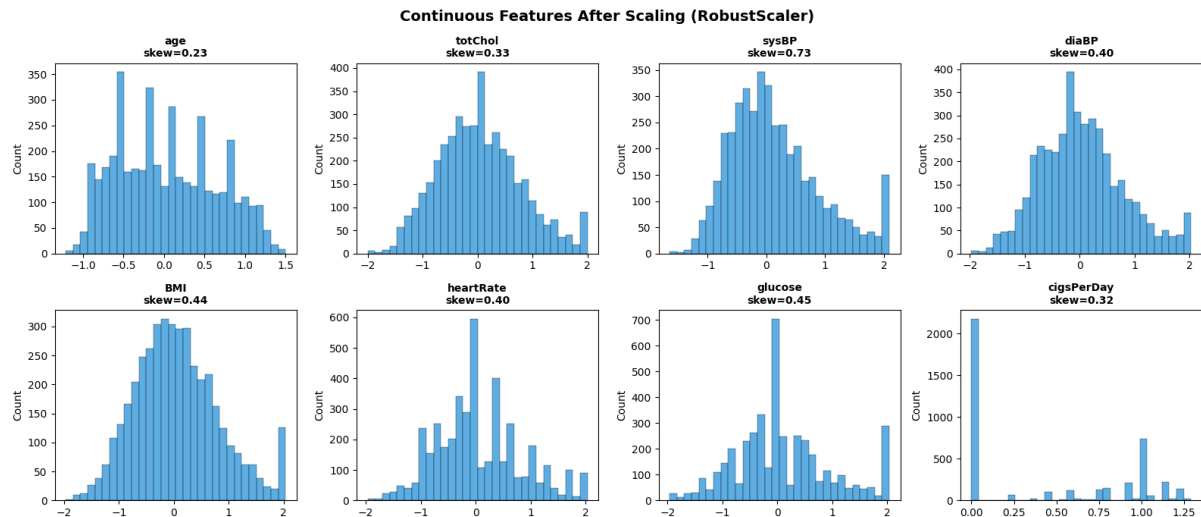
	age	totChol	sysBP	diaBP	BMI	heartRate	glucose	cigsPerDay
count	4238.000	4238.000	4238.000	4238.000	4238.000	4238.000	4238.000	4238.000
mean	0.042	0.039	0.145	0.049	0.064	0.050	0.112	0.441
std	0.612	0.755	0.765	0.766	0.764	0.775	0.881	0.482
min	-1.214	-2.000	-1.648	-1.971	-1.968	-1.967	-1.962	0.000
25%	-0.500	-0.500	-0.407	-0.471	-0.468	-0.467	-0.462	0.000
50%	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000
75%	0.500	0.500	0.593	0.529	0.532	0.533	0.538	1.000
max	1.500	2.000	2.093	2.029	2.032	2.033	2.038	1.291

Distribution after scaling

```
fig, axes = plt.subplots(2, 4, figsize=(16, 7))
axes = axes.flatten()

for i, col in enumerate(scale_cols):
    axes[i].hist(df[col], bins=30, color='#3498db', edgecolor='black', linewidth=0.3, alpha=0.8)
    axes[i].set_title(f"{col}\nskew={df[col].skew():.2f}", fontweight='bold', fontsize=10)
    axes[i].set_ylabel("Count")

plt.suptitle("Continuous Features After Scaling (RobustScaler)", fontsize=14, fontweight='bold')
plt.tight_layout()
plt.show()
```



1. **Scaling preserved distribution shape:** Skewness values remain almost unchanged → RobustScaler scales data but does not normalize distributions.
2. **Outlier robustness achieved:** Features like sysBP and glucose still have tails, but their impact is reduced due to median/IQR-based scaling.
3. **Centered features:** Most variables are now centered around ~ 0 with comparable ranges → improves convergence for ML models (especially gradient-based ones).
4. **Previously skewed feature improved:** cigsPerDay now has acceptable skew (~ 0.32) after log transform → preprocessing pipeline is effective.
5. **No distortion of relationships:** Relative spacing between values is preserved → important for maintaining feature interpretability.
6. **Model readiness:** Data is now well-conditioned for linear/logistic regression and distance-based models (e.g., KNN, SVM).

```
feature_cols = binary_cols + ordinal_cols + scale_cols
df_final = df[feature_cols + [target_col]]

print("Final dataset shape:", df_final.shape)
print("\nColumn list:")
for c in df_final.columns:
    print(f" {c}")
print()
df_final.head()
```

Final dataset shape: (4238, 16)

Column list:

```
male
currentSmoker
BPMeds
prevalentStroke
prevalentHyp
diabetes
education
age
totChol
sysBP
diaBP
BMI
heartRate
glucose
cigsPerDay
TenYearCHD
```

	male	currentSmoker	BPMeds	prevalentStroke	prevalentHyp	diabetes	education	age	totChol	sysBP	diaBP	BMI	heartRate	glucose	cigsPerDay	TenYearCHD
0	1	0.0	0.0	0.0	0.0	0.0	4	-0.714286	-0.696429	-0.814815	-0.806723	0.316692	0.333333	-0.076923	0.000000	0
1	0	0.0	0.0	0.0	0.0	0.0	2	-0.214286	0.285714	-0.259259	-0.067227	0.671710	1.333333	-0.153846	0.000000	0
2	1	1.0	0.0	0.0	0.0	0.0	1	-0.071429	0.196429	-0.018519	-0.134454	-0.012103	0.000000	-0.615385	1.000000	0
3	0	1.0	0.0	0.0	1.0	0.0	3	0.857143	-0.160714	0.814815	0.873950	0.641452	-0.666667	1.923077	1.127923	1
4	0	1.0	0.0	0.0	0.0	0.0	3	-0.214286	0.910714	0.074074	0.134454	-0.463944	0.666667	0.538462	1.043860	0

Resources and References:

- <https://www.geeksforgeeks.org/machine-learning/feature-engineering-scaling-normalization-and-standardization/>
- <https://www.geeksforgeeks.org/data-science/program-find-skewness-statistical-data/>
- <https://www.geeksforgeeks.org/machine-learning/categorical-data-encoding-techniques-in-machine-learning/>

Week No: 7

Dimensionality Reduction

Objectives:

To reduce the dimensionality of a dataset by implementing feature selection techniques and applying Principal Component Analysis (PCA), with the goal of improving model efficiency and reducing complexity. Additionally, to analyze and visualize the variance explained by principal components in order to identify the most significant features contributing to the data.

Work done:

Gokulnaath M – 24BAD028, Nishanth P – 24BAD405

Feature Selection

Method	Type	What it measures
Point-Biserial Correlation	Statistical	Linear association between continuous feature and binary target
Mutual Information	Statistical	Non-linear dependency (captures any relationship, not just linear)
Random Forest Importance	Model-based	Contribution of each feature to reducing Gini impurity across 100 trees

Biserial Correlation with Target

```
corr_scores = {}
for col in feature_cols:
    r, p = pointbiserialr(y, X[col])
    corr_scores[col] = abs(r) # use absolute value - direction doesn't matter for ranking

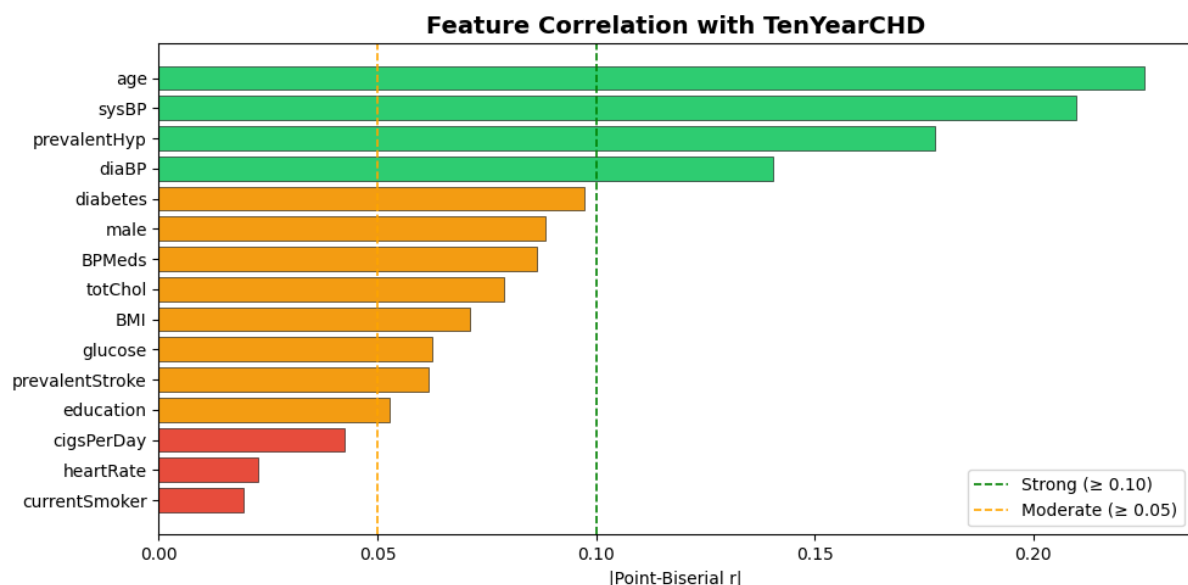
corr_df = pd.DataFrame({"feature": list(corr_scores.keys()),
                        "corr": list(corr_scores.values())}).sort_values("corr", ascending=False)
print(corr_df.to_string(index=False))
```

```
feature    corr
age  0.225256
sysBP 0.209686
prevalentHyp 0.177603
diaBP 0.140473
diabetes 0.097317
male 0.088428
BPMeds 0.086417
totChol 0.079056
BMI 0.071243
glucose 0.062400
prevalentStroke 0.061810
education 0.052812
cigsPerDay 0.042602
heartRate 0.022828
currentSmoker 0.019456
```

1. **Top predictive features (linear relationship):** age, sysBP, and prevalentHyp have the highest correlations → strongest individual predictors of CHD.
2. **Moderate contributors:** diaBP, diabetes, and male show weaker but still meaningful relationships → useful in combination with other features.
3. **Low-impact features:** cigsPerDay, heartRate, and currentSmoker have very low correlation → poor standalone predictors despite domain importance.
4. **Key insight (important):** All correlations are relatively low (<0.25) → CHD prediction is inherently **multivariate**, requiring models to learn interactions rather than relying on single features.

Visualization

```
plt.figure(figsize=(10, 5))
bars = plt.barh(corr_df["feature"][:, -1], corr_df["corr"][:, -1],
                color=[ '#2ecc71' if v ≥ 0.1 else '#f39c12' if v ≥ 0.05 else '#e74c3c'
                        for v in corr_df["corr"][:, -1]],
                edgecolor='black', linewidth=0.4)
plt.axvline(0.10, color='green', linestyle='--', linewidth=1.2, label='Strong (≥ 0.10)')
plt.axvline(0.05, color='orange', linestyle='--', linewidth=1.2, label='Moderate (≥ 0.05)')
plt.xlabel("|Point-Biserial r|")
plt.title("Feature Correlation with TenYearCHD", fontsize=14, fontweight='bold')
plt.legend()
plt.tight_layout()
plt.show()
```



1. **Strongest predictors:** age, sysBP, prevalentHyp, and diaBP exceed the strong threshold (≥ 0.10) → primary drivers of CHD risk.

2. **Moderate features:** diabetes, male, BPMeds, totChol, BMI, and glucose fall in the moderate range → useful but not dominant individually.
3. **Weak predictors:** cigsPerDay, heartRate, and currentSmoker have low correlation → minimal standalone impact.
4. **Key takeaway:** Even the strongest features have relatively low correlations (<0.25) → CHD prediction depends on **combined feature interactions**, not single variables.

Mutual Information

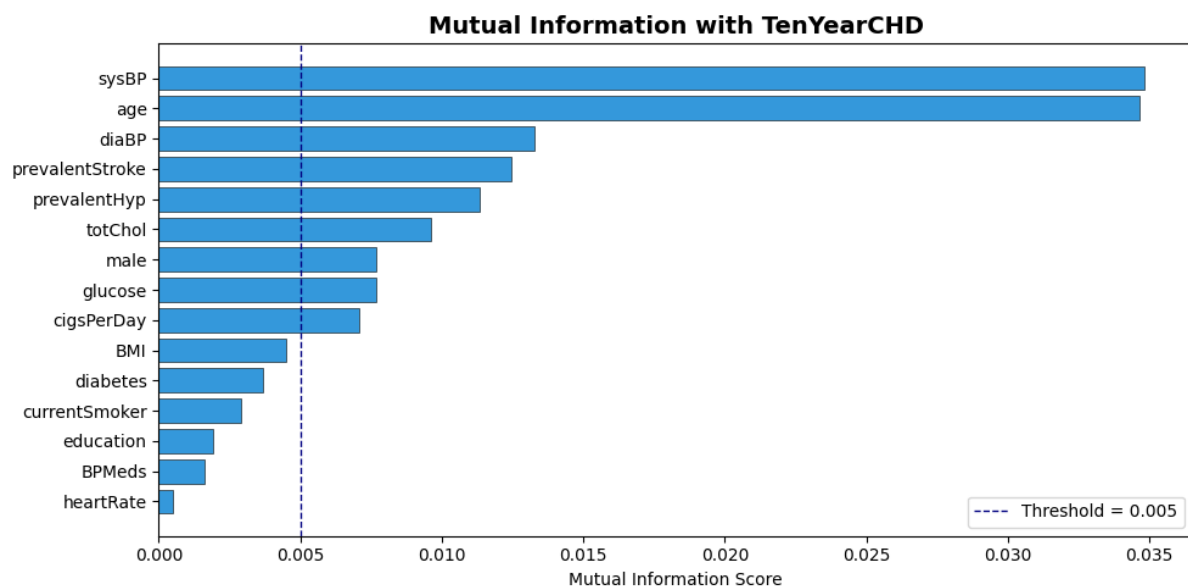
```
mi_scores = mutual_info_classif(X, y, random_state=42)
mi_df = pd.DataFrame({"feature": feature_cols, "MI": mi_scores}).sort_values("MI", ascending=False)
print(mi_df.to_string(index=False))
```

```
feature      MI
sysBP 0.034811
age 0.034659
diaBP 0.013259
prevalentStroke 0.012476
prevalentHyp 0.011348
totChol 0.009608
male 0.007684
glucose 0.007662
cigsPerDay 0.007074
BMI 0.004518
diabetes 0.003688
currentSmoker 0.002898
education 0.001929
BPMeds 0.001620
heartRate 0.000509
```

1. **Top features (nonlinear importance):** sysBP and age have the highest mutual information → they remain the most informative even beyond linear relationships.
2. **BP-related features still strong:** diaBP and prevalentHyp rank high → reinforces that blood pressure dynamics are critical for CHD prediction.
3. **Some features gain relevance vs correlation:** prevalentStroke appears relatively important here despite low correlation → indicates **nonlinear or interaction effects**.
4. **Low MI features truly weak:** heartRate, BPMeds, and education have near-zero MI → minimal contribution even in nonlinear models.
5. **Key insight:** Mutual Information values are small overall → confirms CHD is a **complex, multi-factor problem**, requiring models that capture interactions (e.g., tree-based models, ensembles).

Visualization

```
plt.figure(figsize=(10, 5))
palette = ['#3498db' if v > 0 else '#bdc3c7' for v in mi_df["MI"]]
plt.barh(mi_df["feature"][::-1], mi_df["MI"][::-1],
         color=palette[::-1], edgecolor='black', linewidth=0.4)
plt.xlabel("Mutual Information Score")
plt.title("Mutual Information with TenYearCHD", fontsize=14, fontweight='bold')
plt.axvline(0.005, color='navy', linestyle='--', linewidth=1, label='Threshold = 0.005')
plt.legend()
plt.tight_layout()
plt.show()
```



1. **Top nonlinear drivers:** `sysBP` and `age` clearly dominate → strongest features even when capturing nonlinear relationships.
2. **BP ecosystem remains critical:** `diaBP` and `prevalentHyp` also rank high → confirms cardiovascular pressure variables are core to CHD prediction.
3. **Hidden nonlinear signals:** `prevalentStroke` ranks relatively high despite weak linear correlation → indicates interaction/nonlinear effects missed by Pearson correlation.
4. **Moderate contributors:** `totChol`, `male`, `glucose`, and `cigsPerDay` provide additional signal → useful in combination but not dominant.
5. **Low-value features:** `education`, `BPMeds`, and `heartRate` fall below threshold → negligible contribution even in nonlinear context.
6. **Key takeaway:** Feature importance is distributed → best performance will come from models that **combine multiple weak-to-moderate signals** (e.g., Random Forest, XGBoost), not single-feature thresholds.

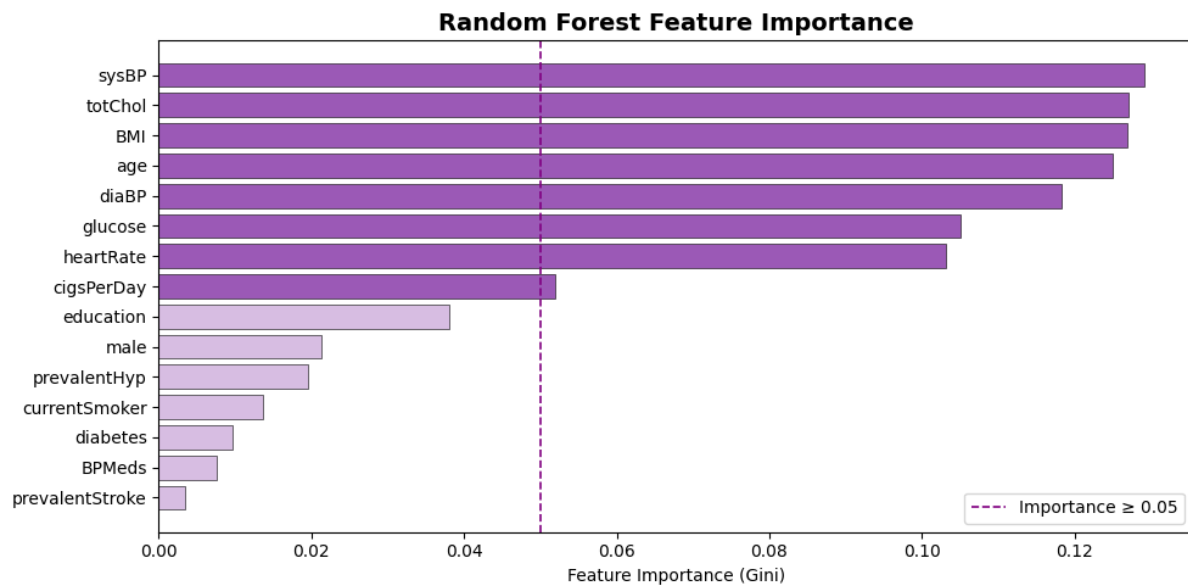
Random Forest Feature Importance

```
rf = RandomForestClassifier(n_estimators=100, random_state=42, n_jobs=-1)
rf.fit(X, y)

rf_df = pd.DataFrame({"feature": feature_cols,
                      "importance": rf.feature_importances_}).sort_values("importance", ascending=False)
print(rf_df.to_string(index=False))
```

feature	importance
sysBP	0.129061
totChol	0.127067
BMI	0.126883
age	0.125064
diaBP	0.118283
glucose	0.105078
heartRate	0.103098
cigsPerDay	0.051976
education	0.038001
male	0.021313
prevalentHyp	0.019595
currentSmoker	0.013721
diabetes	0.009724
BPMeds	0.007635
prevalentStroke	0.003499

1. **Top contributors (tree-based importance):** sysBP, totChol, BMI, and age are the most important → Random Forest captures both linear and nonlinear effects, elevating features like BMI and cholesterol.
2. **Broader feature utilization:** Unlike correlation/MI, importance is more evenly distributed → tree models leverage **interactions across many features**, not just top ones.
3. **Shift vs earlier methods:** BMI, glucose, and heartRate gain importance here despite low correlation → indicates **nonlinear relationships and feature interactions**.
4. **Lower importance ≠ useless:** Features like prevalentHyp, diabetes, and male appear lower but still contribute in splits → tree models distribute importance across correlated variables.
5. **Multicollinearity effect:** Highly correlated features (sysBP, diaBP, prevalentHyp) share importance → reduces individual scores but not overall contribution.
6. **Key insight:** Random Forest confirms CHD prediction is **not dominated by a single feature** → best performance comes from combining multiple moderately important variables.



1. **Dominant features (≥ 0.05):** `sysBP`, `totChol`, `BMI`, `age`, `diaBP`, `glucose`, and `heartRate` form the core predictive set → strongest contributors in the model.
2. **Top driver:** `sysBP` ranks highest → confirms systolic blood pressure as the most influential feature across all analyses.
3. **Nonlinear importance revealed:** `BMI`, `glucose`, and `heartRate` are highly ranked here despite weak correlation earlier → tree models capture hidden interactions.
4. **Mid-tier feature:** `cigsPerDay` sits near the threshold → smoking intensity contributes moderately when modeled nonlinearly.
5. **Lower-tier features:** `education`, `male`, `prevalentHyp`, and `currentSmoker` have limited standalone impact → secondary role.
6. **Minimal contributors:** `diabetes`, `BPMeds`, and `prevalentStroke` show very low importance → weak influence in this model setup.
7. **Key insight:** Feature importance is distributed across many variables → Random Forest relies on **ensemble of weak-to-moderate predictors**, not a single dominant feature.

Consensus Ranking — Combining All Three Methods

```
# Rank each method (1 = best)
corr_rank = corr_df.set_index("feature")["corr"].rank(ascending=False)
mi_rank    = mi_df.set_index("feature")["MI"].rank(ascending=False)
rf_rank    = rf_df.set_index("feature")["importance"].rank(ascending=False)

consensus = pd.DataFrame({
    "Corr_Rank": corr_rank,
    "MI_Rank"   : mi_rank,
    "RF_Rank"   : rf_rank
})
consensus["Avg_Rank"] = consensus.mean(axis=1)
consensus = consensus.sort_values("Avg_Rank")
consensus["Avg_Rank"] = consensus["Avg_Rank"].round(2)

print("Consensus Feature Ranking (lower = more important):\n")
print(consensus.to_string())
```

Consensus Feature Ranking (lower = more important):

feature	Corr_Rank	MI_Rank	RF_Rank	Avg_Rank
sysBP	2.0	1.0	1.0	1.33
age	1.0	2.0	4.0	2.33
diaBP	4.0	3.0	5.0	4.00
totChol	8.0	6.0	2.0	5.33
prevalentHyp	3.0	5.0	11.0	6.33
BMI	9.0	10.0	3.0	7.33
male	6.0	7.0	10.0	7.67
glucose	10.0	8.0	6.0	8.00
diabetes	5.0	11.0	13.0	9.67
prevalentStroke	11.0	4.0	15.0	10.00
cigsPerDay	13.0	9.0	8.0	10.00
education	12.0	13.0	9.0	11.33
BPMeds	7.0	14.0	14.0	11.67
heartRate	14.0	15.0	7.0	12.00
currentSmoker	15.0	12.0	12.0	13.00

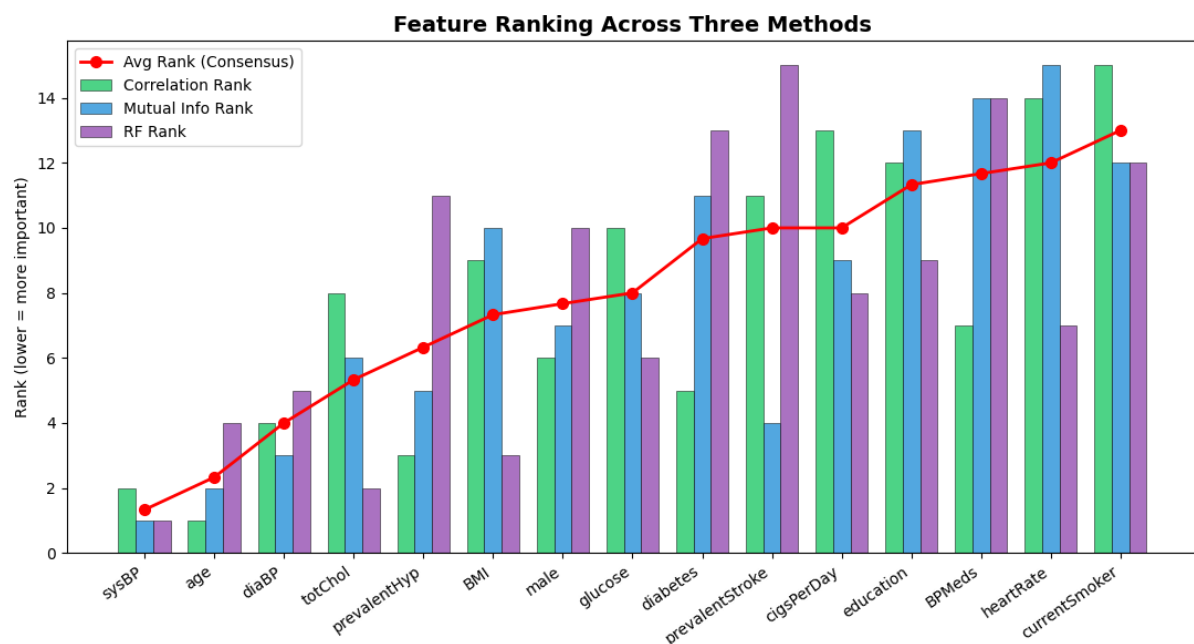
1. **Consistent top features across methods:** `sysBP` and `age` rank highest overall → robust predictors regardless of method (linear, nonlinear, tree-based).
2. **Core clinical set:** `diaBP`, `totChol`, and `prevalentHyp` consistently appear in upper ranks → key cardiovascular risk indicators.
3. **Model-dependent features:** `BMI` and `glucose` rank higher in Random Forest but lower in correlation → importance comes from nonlinear interactions.
4. **Weak and unstable features:** `currentSmoker`, `heartRate`, `BPMeds`, and `education` rank low across methods → minimal contribution to prediction.

```
fig, ax = plt.subplots(figsize=(11, 6))
x = np.arange(len(consensus))
width = 0.25

ax.bar(x - width, consensus["Corr_Rank"], width, label="Correlation Rank", color="#2ecc71", alpha=0.85, edgecolor='black', linewidth=0.4)
ax.bar(x, consensus["MI_Rank"], width, label="Mutual Info Rank", color="#3498db", alpha=0.85, edgecolor='black', linewidth=0.4)
ax.bar(x + width, consensus["RF_Rank"], width, label="RF Rank", color="#9b59b6", alpha=0.85, edgecolor='black', linewidth=0.4)

ax.plot(x, consensus["Avg_Rank"], 'r-o', linewidth=2, markersize=7, label="Avg Rank (Consensus)", zorder=5)

ax.set_xticks(x)
ax.set_xticklabels(consensus.index, rotation=35, ha='right')
ax.set_ylabel("Rank (lower = more important)")
ax.set_title("Feature Ranking Across Three Methods", fontsize=14, fontweight='bold')
ax.legend()
plt.tight_layout()
plt.show()
```



1. **Stable top predictors:** `sysBP`, `age`, and `diaBP` consistently rank high across all three methods → strong, reliable features.
2. **Method variability reveals interactions:** Features like `BMI`, `totChol`, and `cigPerDay` fluctuate across methods → importance depends on whether the model captures linear vs nonlinear effects.
3. **Mid-tier but useful features:** `prevalentHyp`, `male`, and `glucose` stay in the middle → contribute in combination rather than alone.
4. **Consistently weak features:** `currentSmoker`, `heartRate`, `BPMeds`, and `education` rank low across methods → limited predictive value.

```
selected_features = list(consensus.index[:10])
dropped_features = list(consensus.index[10:])

print("✅ Selected features:", selected_features)
print()
print("❌ Dropped features :", dropped_features)

X_selected = X[selected_features]
print(f"\nReduced from {X.shape[1]} → {X_selected.shape[1]} features")

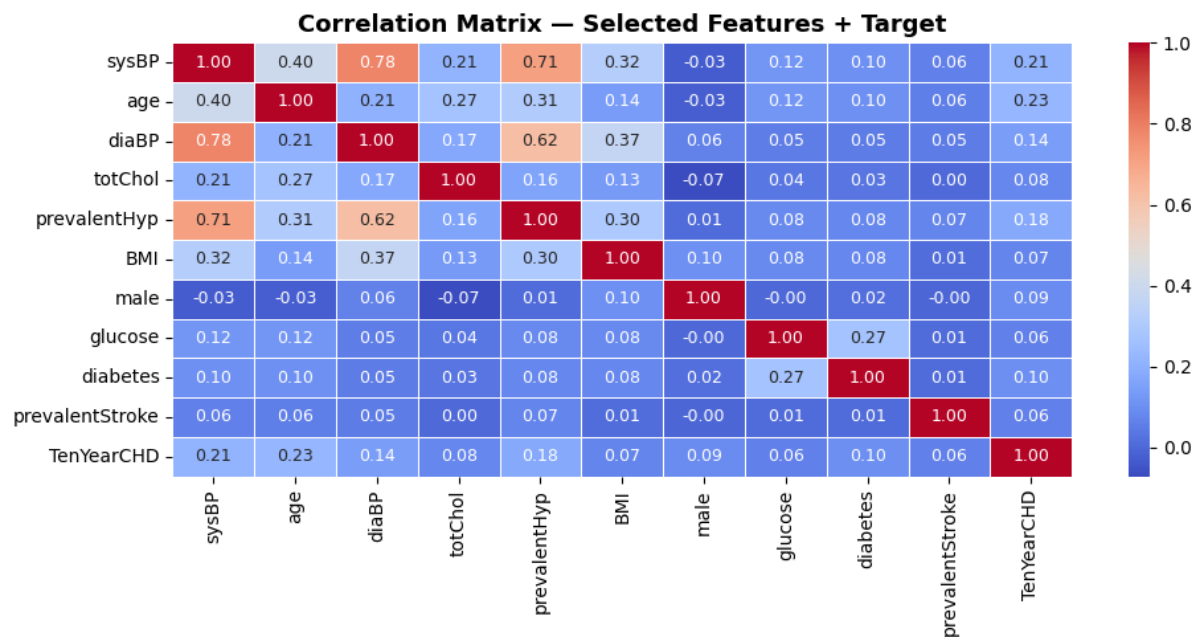
✅ Selected features: ['sysBP', 'age', 'diaBP', 'totChol', 'prevalentHyp', 'BMI', 'male', 'glucose', 'diabetes', 'prevalentStroke']
❌ Dropped features : ['cigsPerDay', 'education', 'BPMeds', 'heartRate', 'currentSmoker']

Reduced from 15 → 10 features
```

1. **Strong feature set retained:** Selected features (sysBP, age, diaBP, totChol, prevalentHyp, etc.) align with top ranks across all methods → solid, evidence-based selection.
2. **Dimensionality reduced effectively:** Reduced from 15 → 10 features → lowers model complexity and risk of overfitting without losing key signal.
3. **Dropped features are weak:** Removed features (currentSmoker, heartRate, education, etc.) consistently ranked low → minimal information loss.
4. **Good balance achieved:** Final set includes demographic, clinical, and medical history features → ensures model captures both direct and interaction effects.

```
# Heatmap of selected features vs target
plt.figure(figsize=(10, 5))
corr_matrix = pd.concat([X_selected, y], axis=1).corr()
mask = np.zeros_like(corr_matrix, dtype=bool)

sns.heatmap(corr_matrix, annot=True, fmt=".2f", cmap='coolwarm',
            linewidths=0.5, annot_kws={"size": 9})
plt.title("Correlation Matrix - Selected Features + Target", fontsize=13, fontweight='bold')
plt.tight_layout()
plt.show()
```



- Moderate correlation with target:** age (~0.23) and sysBP (~0.21) are the strongest linear predictors of CHD, followed by prevalentHyp (~0.18) and diaBP (~0.14).
- Multicollinearity present:** Strong correlations among BP-related features (sysBP-diaBP ~0.78, sysBP-prevalentHyp ~0.71) → potential issue for linear models.
- Weak individual signals:** Features like BMI, glucose, male, and prevalentStroke show low correlation with CHD → limited standalone impact.
- Independent features:** Some variables (e.g., male, glucose) have low correlation with others → useful for adding diverse information to the model.
- Key insight:** No feature has high correlation with CHD (>0.3) → confirms prediction depends on **combination of features**, not a single dominant variable.

Kamalesh N – 24BAD054, Jayaraksha Reguraj – 24BAD044

PCA

PCA transforms the selected features into a new set of **uncorrelated axes**(Principal Components) ordered by the amount of variance they explain.

How many dimensions do we actually need to capture most of the information in this dataset?

```
pca = PCA(random_state=42)
pca.fit(X_selected)

explained_var = pca.explained_variance_ratio_
cumulative_var = np.cumsum(explained_var)

pca_summary = pd.DataFrame({
    "Component"      : [f"PC{i+1}" for i in range(len(explained_var))],
    "Explained Var %" : (explained_var * 100).round(2),
    "Cumulative %"    : (cumulative_var * 100).round(2)
})
print(pca_summary.to_string(index=False))
```

Component	Explained Var %	Cumulative %
PC1	35.80	35.80
PC2	19.21	55.00
PC3	13.92	68.93
PC4	11.70	80.63
PC5	7.61	88.24
PC6	6.02	94.26
PC7	2.93	97.19
PC8	2.08	99.28
PC9	0.58	99.85
PC10	0.15	100.00

1. **Variance concentrated in few components:** First 3 PCs explain ~69% variance → most information is captured early.
2. **Good dimensionality reduction point:** ~80% variance is reached by PC4 → you can safely reduce from 10 → 4 features with minimal information loss.
3. **Diminishing returns after PC6:** Beyond PC6 (~94%), each additional component adds very little → not worth the complexity.
4. **Key insight (important):** PCA confirms redundancy among features (especially BP-related) → multiple original features are compressible into fewer latent components.

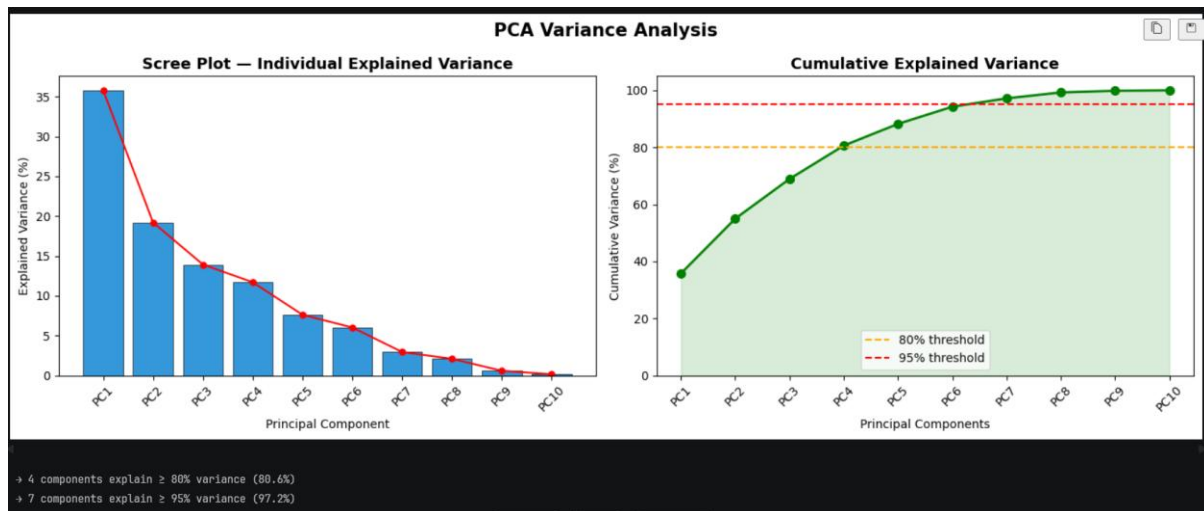
```
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))

# --- Scree Plot ---
components = [f"PC{i+1}" for i in range(len(explained_var))]
ax1.bar(components, explained_var * 100, color='█'#3498db', edgecolor='black', linewidth=0.5)
ax1.plot(components, explained_var * 100, 'r-o', linewidth=1.5, markersize=5)
ax1.set_title("Scree Plot – Individual Explained Variance", fontsize=13, fontweight='bold')
ax1.set_xlabel("Principal Component")
ax1.set_ylabel("Explained Variance (%)")
ax1.set_xticklabels(components, rotation=45)

# --- Cumulative Variance ---
ax2.plot(components, cumulative_var * 100, 'g-o', linewidth=2, markersize=7)
ax2.axhline(80, color='orange', linestyle='--', linewidth=1.5, label='80% threshold')
ax2.axhline(95, color='red', linestyle='--', linewidth=1.5, label='95% threshold')
ax2.fill_between(range(len(components)), cumulative_var * 100, alpha=0.15, color='green')
ax2.set_xticks(range(len(components)))
ax2.set_xticklabels(components, rotation=45)
ax2.set_title("Cumulative Explained Variance", fontsize=13, fontweight='bold')
ax2.set_xlabel("Principal Components")
ax2.set_ylabel("Cumulative Variance (%)")
ax2.legend()
ax2.set_ylim(0, 105)

plt.suptitle("PCA Variance Analysis", fontsize=15, fontweight='bold')
plt.tight_layout()
plt.show()
```

```
# Report threshold
for n, cv in enumerate(cumulative_var, 1):
    if cv ≥ 0.80:
        print(f" → {n} components explain ≥ 80% variance ({cv*100:.1f}%)")
        break
for n, cv in enumerate(cumulative_var, 1):
    if cv ≥ 0.95:
        print(f" → {n} components explain ≥ 95% variance ({cv*100:.1f}%)")
        break
```



1. **Clear elbow at PC3–PC4:** After PC3/PC4, the explained variance drops sharply → optimal cutoff point for dimensionality reduction.
2. **Efficient compression:**
 - 4 components → ~80% variance
 - 7 components → ~97% variance
→ strong evidence of feature redundancy.
3. **First component dominance:** PC1 alone explains ~36% → indicates a strong underlying combined factor (likely BP + age effects).
4. **Practical takeaway:**
 - Use **4 PCs** for simpler models (good trade-off)
 - Use **6–7 PCs** if you want near-complete information retention
5. **Critical thinking point (don't miss this):** PCA improves efficiency but **loses interpretability** → original features (like sysBP, age) are more meaningful in real-world/medical context.

PCA Component Loading Heatmap

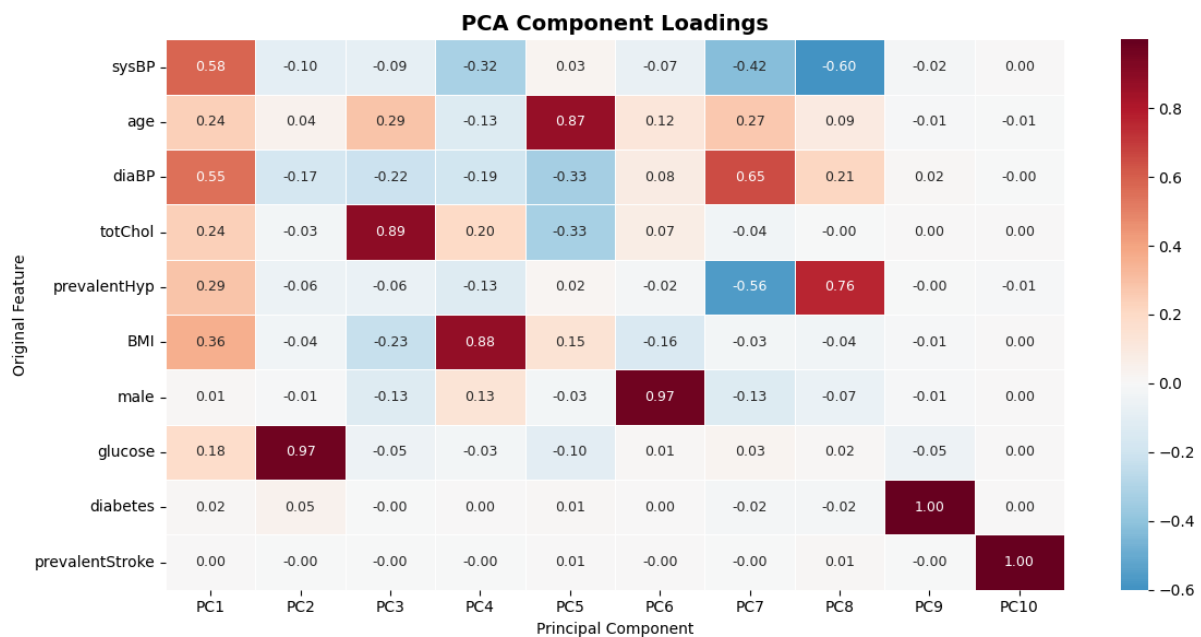
```

loadings = pd.DataFrame(
    pca.components_.T,
    index=selected_features,
    columns=[f"PC{i+1}" for i in range(len(explained_var))]
)

plt.figure(figsize=(12, 6))
sns.heatmap(loadings, annot=True, fmt=".2f", cmap='RdBu_r',
            center=0, linewidths=0.5, annot_kws={"size": 9})
plt.title("PCA Component Loadings", fontsize=14, fontweight='bold')
plt.ylabel("Original Feature")
plt.xlabel("Principal Component")
plt.tight_layout()
plt.show()

print("Interpretation:")
for i in range(min(3, len(explained_var))):
    top = loadings[f"PC{i+1}"].abs().nlargest(3).index.tolist()
    print(f" PC{i+1} ({explained_var[i]*100:.1f}% var): mainly driven by {top}")

```



Interpretation:

PC1 (35.8% var): mainly driven by ['sysBP', 'diaBP', 'BMI']

PC2 (19.2% var): mainly driven by ['glucose', 'diaBP', 'sysBP']

PC3 (13.9% var): mainly driven by ['totChol', 'age', 'BMI']

1. **PC1 = Cardiovascular pressure factor:** High loadings on `sysBP` (0.58), `diaBP` (0.55), and `BMI` → represents overall **blood pressure + body condition** axis (most important component).
2. **PC2 = Glucose-driven factor:** Dominated by `glucose` (0.97) → captures metabolic/diabetic signal independently from other features.
3. **PC3 = Cholesterol + age factor:** Strong loading on `totChol` (0.89) with contribution from `age` → represents **lipid + aging risk**.
4. **Feature grouping insight:** PCA separates variables into meaningful clusters:
 - BP group (`sysBP`, `diaBP`, `prevalentHyp`)
 - Metabolic group (`glucose`, `diabetes`)
 - Lipid/age group (`totChol`, `age`)→ confirms underlying physiological structure in data.
5. **Redundancy confirmed:** Similar features (e.g., `sysBP`, `diaBP`, `prevalentHyp`) load together → explains why fewer PCs capture most variance.
6. **Critical limitation:** PCs are linear combinations → while efficient, they lose direct interpretability (you can't say "risk increases by X BP" anymore).

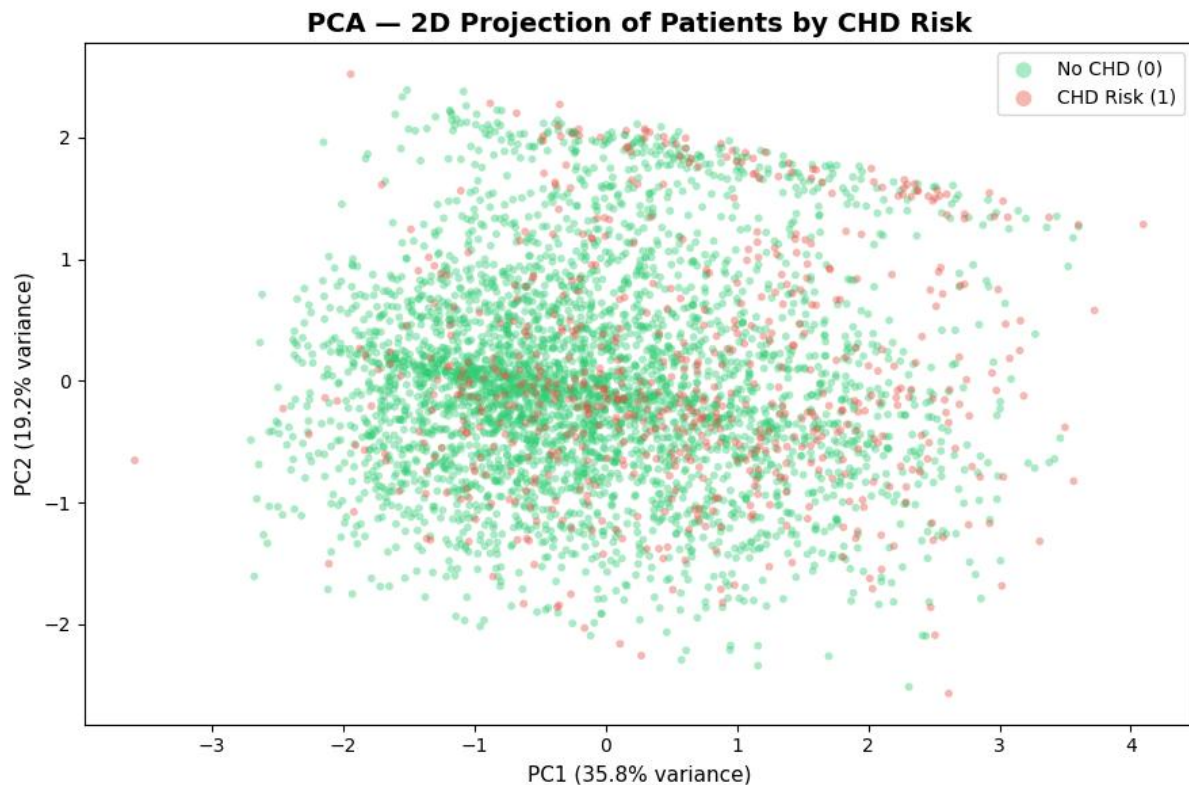
2D PCA Scatter

```
X_pca = pca.transform(X_selected)

pca2d = pd.DataFrame({"PC1": X_pca[:, 0], "PC2": X_pca[:, 1], "TenYearCHD": y.values})

plt.figure(figsize=(9, 6))
colors = {0: '#2ecc71', 1: '#e74c3c'}
labels = {0: 'No CHD (0)', 1: 'CHD Risk (1)'}
for cls in [0, 1]:
    sub = pca2d[pca2d["TenYearCHD"] == cls]
    plt.scatter(sub["PC1"], sub["PC2"], c=colors[cls], label=labels[cls],
                alpha=0.4, s=18, edgecolors='none')

plt.xlabel(f"PC1 ({explained_var[0]*100:.1f}% variance)", fontsize=11)
plt.ylabel(f"PC2 ({explained_var[1]*100:.1f}% variance)", fontsize=11)
plt.title("PCA - 2D Projection of Patients by CHD Risk", fontsize=14, fontweight='bold')
plt.legend(markerscale=2)
plt.tight_layout()
plt.show()
```



1. **No clear class separation:** CHD (red) and non-CHD (green) points heavily overlap → PCA (unsupervised) does not separate classes well.
2. **Weak signal in top components:** Even with ~55% variance (PC1 + PC2), class distinction is not visible → CHD signal is not aligned with maximum variance directions.
3. **Slight trend at extremes:** Some CHD cases appear more toward higher PC1 values → indicates weak directional pattern but not strong enough for separation.
4. **Key insight (important):** PCA is **not suitable for classification directly** → it captures variance, not class boundaries. Use supervised models (Logistic Regression, Random Forest, XGBoost) for prediction.

3D PCA Scatter

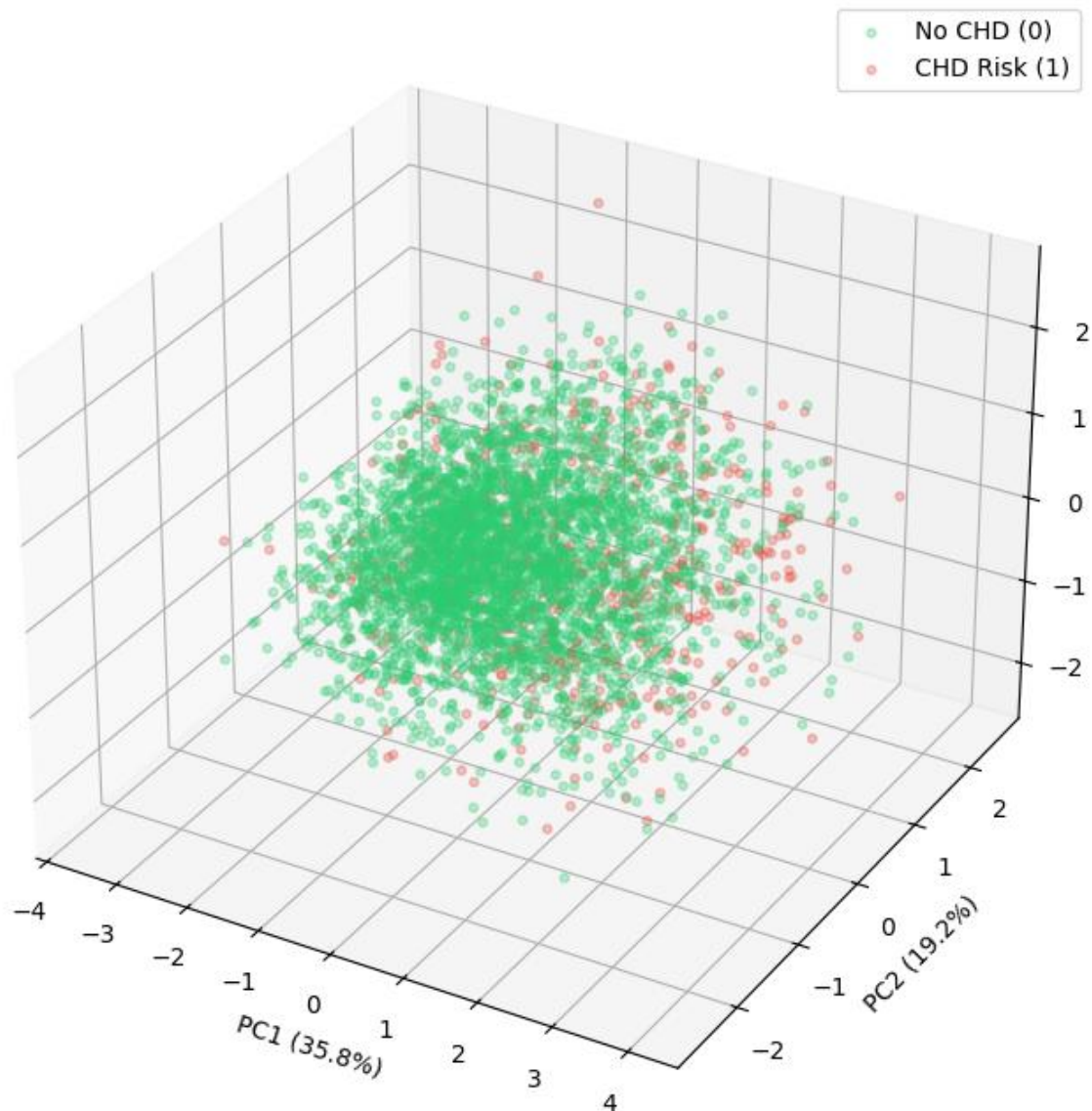
```
from mpl_toolkits.mplot3d import Axes3D # noqa

fig = plt.figure(figsize=(10, 7))
ax = fig.add_subplot(111, projection='3d')

for cls in [0, 1]:
    mask = y.values == cls
    ax.scatter(X_pca[mask, 0], X_pca[mask, 1], X_pca[mask, 2],
               c=colors[cls], label=labels[cls], alpha=0.3, s=12)

ax.set_xlabel(f"PC1 ({explained_var[0]*100:.1f}%)")
ax.set_ylabel(f"PC2 ({explained_var[1]*100:.1f}%)")
ax.set_zlabel(f"PC3 ({explained_var[2]*100:.1f}%)")
ax.set_title("PCA - 3D Projection by CHD Risk", fontsize=13, fontweight='bold')
ax.legend()
plt.tight_layout()
plt.show()
```

PCA — 3D Projection by CHD Risk



1. **Still no clear separation in 3D:** Even with PC1–PC3 (~69% variance), CHD and non-CHD points remain highly overlapping → adding dimensions does not resolve class separation.
2. **Variance ≠ class information:** PCA captures directions of maximum variance, not class-discriminative directions → important limitation for classification tasks.
3. **Slight clustering tendency:** CHD points appear slightly more dispersed toward outer regions, but no distinct boundary exists → weak signal at best.
4. **Conclusion (critical):** PCA is useful for **dimensionality reduction**, not for classification → rely on supervised models (e.g., Logistic Regression, Random Forest, XGBoost) to learn decision boundaries.

Choosing Number of Component

8 components to retain $\geq 95\%$ of variance

```

n_components = 9 # covers ≥ 95% cumulative variance

pca_final = PCA(n_components=n_components, random_state=42)
X_pca_final = pca_final.fit_transform(X_selected)

print(f"Original feature count : {X_selected.shape[1]}")
print(f"PCA components kept      : {n_components}")
print(f"Cumulative variance      : {pca_final.explained_variance_ratio_.sum()*100:.2f}%")
print(f"Variance lost             : {(1 - pca_final.explained_variance_ratio_.sum())*100:.2f}%")

pca_cols = [f"PC{i+1}" for i in range(n_components)]
df_pca = pd.DataFrame(X_pca_final, columns=pca_cols)
df_pca["TenYearCHD"] = y.values
df_pca.head()

```

```

Original feature count : 10
PCA components kept    : 9
Cumulative variance    : 99.85%
Variance lost          : 0.15%

```

```

Original feature count : 10
PCA components kept    : 9
Cumulative variance    : 99.85%
Variance lost          : 0.15%

```

	PC1	PC2	PC3	PC4	PC5	PC6	PC7	PC8	PC9	TenYearCHD
0	-1.411708	0.051262	-0.698540	0.770817	-0.119695	0.374720	-0.234280	0.041787	-0.009011	0
1	-0.223107	-0.218900	0.156923	0.759374	-0.153544	-0.499527	0.226509	-0.039161	-0.005496	0
2	-0.429422	-0.655793	0.160825	0.203390	-0.051687	0.562035	-0.004289	-0.234574	0.006940	0
3	1.723267	1.539504	-0.390587	-0.211922	0.439022	-0.405849	0.176507	0.409431	-0.122616	1
4	-0.052011	0.410277	0.864262	-0.279198	-0.651168	-0.281381	0.241128	-0.145213	-0.030684	0

1. **Near-lossless transformation:** 9 PCs retain **99.85% variance** → PCA has preserved almost all information (only 0.15% loss).
2. **Minimal dimensionality reduction:** Reduced from 10 → 9 features → technically valid, but **not efficient** (very small compression gain).
3. **Trade-off concern (important):** You sacrificed interpretability (original features like `sysBP`, `age`) but gained almost nothing in dimensionality reduction → **not a practical choice**.
4. **Better strategy:**
 - Use **4–6 PCs** (80–95% variance) for real reduction
 - Or skip PCA entirely and use original features (since only 10 features already)
5. **Engineering judgment (this matters):** PCA is useful when **feature count is high (100+)** → here, feature selection already did the job better than PCA.

Resources and References:

- <https://www.geeksforgeeks.org/machine-learning/feature-selection-techniques-in-machine-learning/>

- <https://www.geeksforgeeks.org/data-analysis/principal-component-analysis-pca/>

Week No: 8

Dashboard Construction

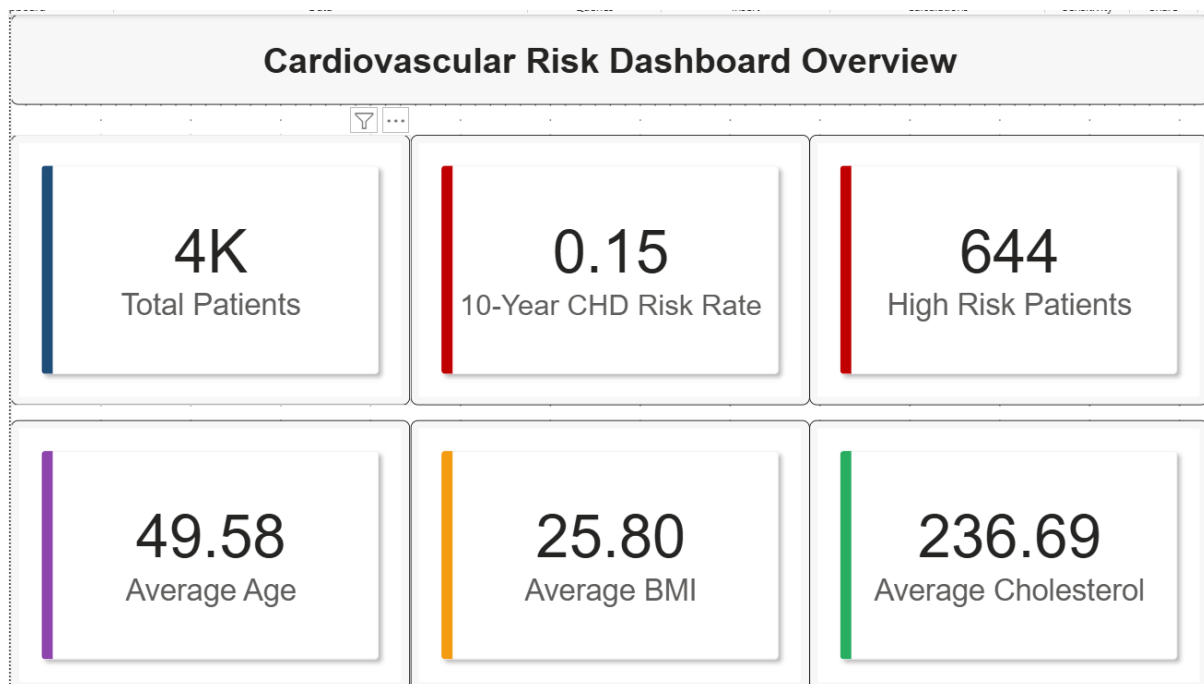
Objectives:

To design and develop an interactive and insightful dashboard using Power BI that effectively integrates diverse data visuals into a cohesive narrative, enabling users to easily interpret trends, derive actionable insights, and support informed decision-making.

Work done:

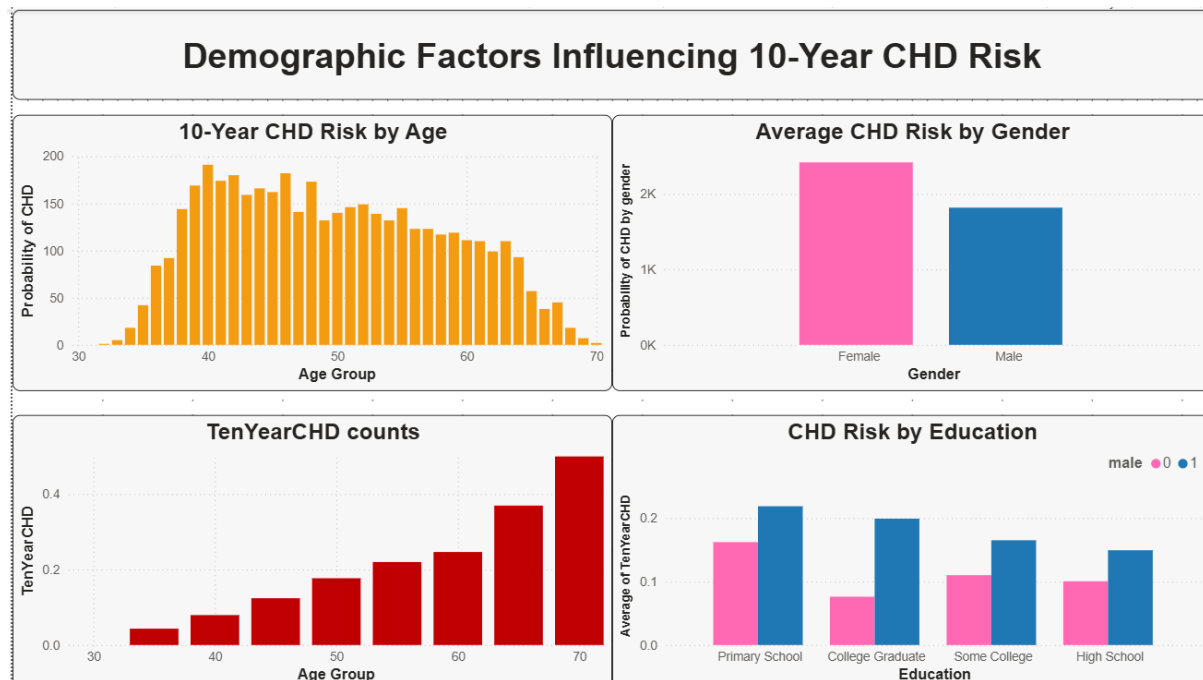
Kamalesh N – 24BAD054, Jayaraksha Reguraj – 24BAD044

KPI Cards



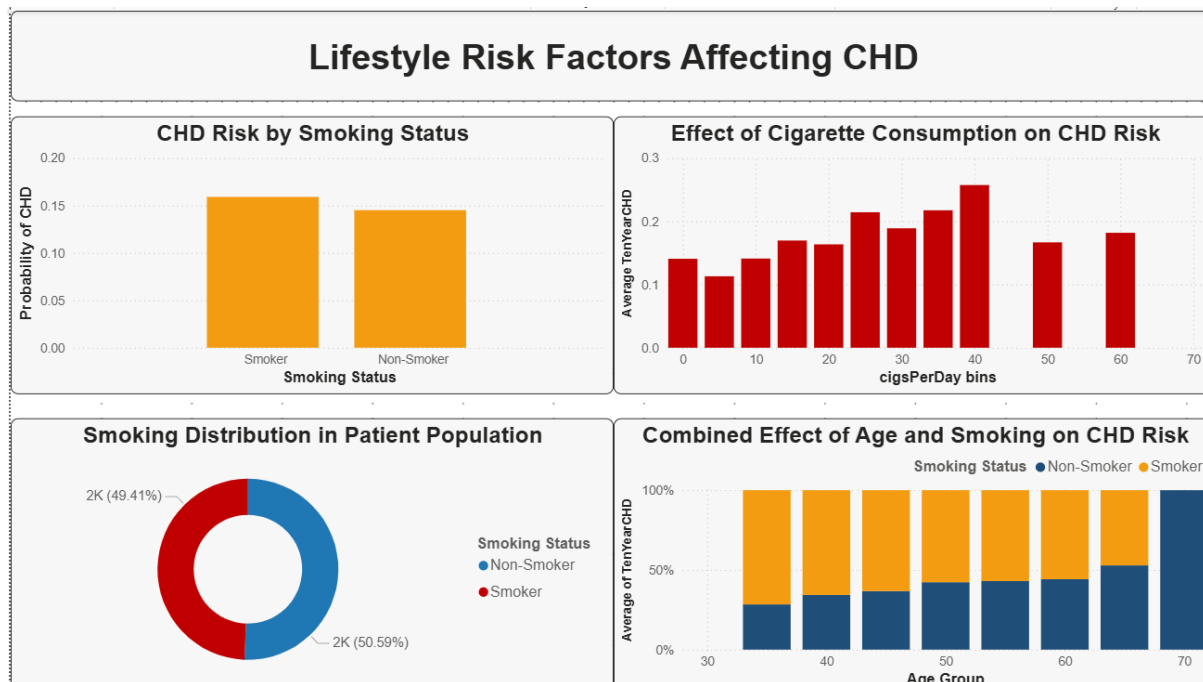
1. **Moderate overall CHD risk:** ~15% 10-year risk (644 out of ~4K patients) → non-trivial proportion; warrants preventive interventions at population level.
2. **Risk profile aligns with key drivers:** Average age (~50), BMI (~25.8), and cholesterol (~236) are in ranges associated with elevated CHD risk → consistent with earlier feature importance (age, BP, cholesterol).
3. **Clinical focus areas:** Given prior analysis, targeting **blood pressure control + cholesterol management + aging population screening** will likely yield the highest impact on reducing CHD cases.

Demographic Risk Factors



1. **Age is the dominant risk driver:** CHD risk rises steadily with age, peaking in older groups (~60+) → strongest demographic predictor.
2. **Gender difference visible but needs normalization:** Raw counts show females higher, but this is likely due to sample size imbalance → use **rate (mean risk)**, not counts, for valid comparison.
3. **Education shows inverse trend (weak signal):** Lower education groups tend to have higher CHD risk → possible socioeconomic/behavioral influence, but effect is modest.

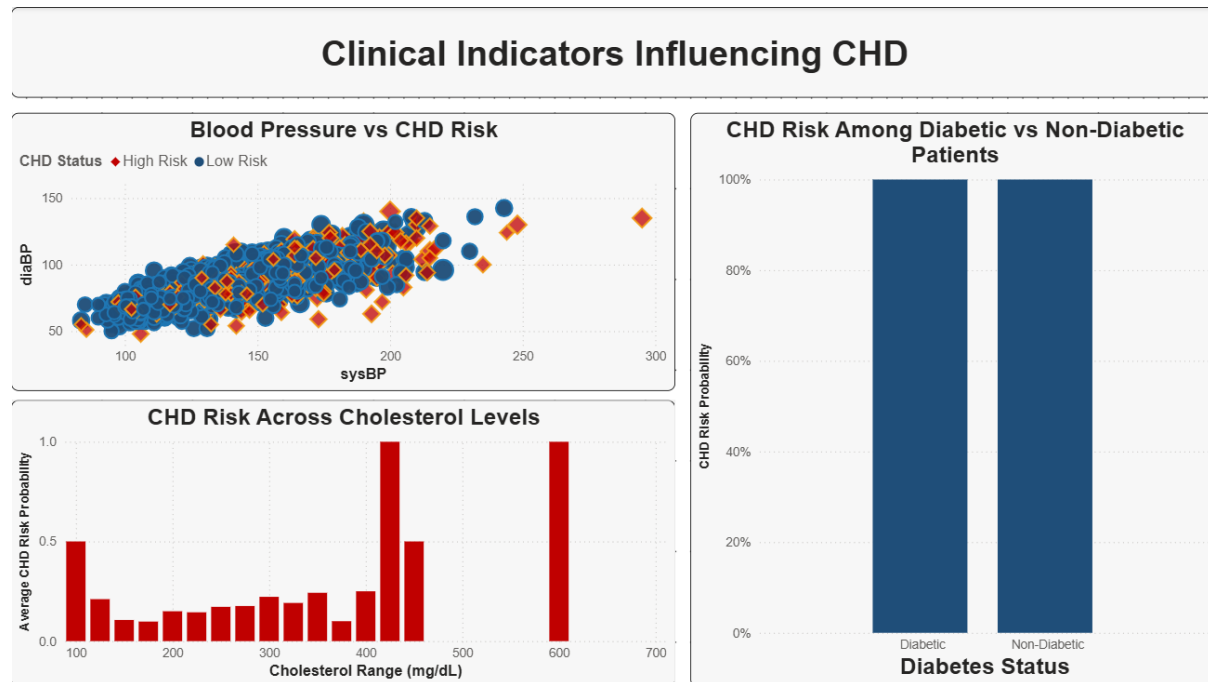
Life Style Risk



1. **Smoking effect is present but moderate:** Smokers have slightly higher CHD risk (~16% vs ~14.5%), but the gap is not large → smoking alone is not a dominant predictor.
2. **Dose-response relationship exists:** CHD risk increases with `cigsPerDay` → heavier smoking (30–40+) clearly raises risk → intensity matters more than binary smoking status.
3. **Population insight:** Nearly equal split between smokers and non-smokers → smoking is widespread, making it an important **population-level risk factor** despite moderate individual effect.
4. **Interaction with age is critical:** CHD risk increases significantly with age for both groups, but smokers consistently show higher risk → **smoking amplifies age-related risk**, not replaces it.

Gokulnaath M – 24BAD028, Nishanth P – 24BAD405

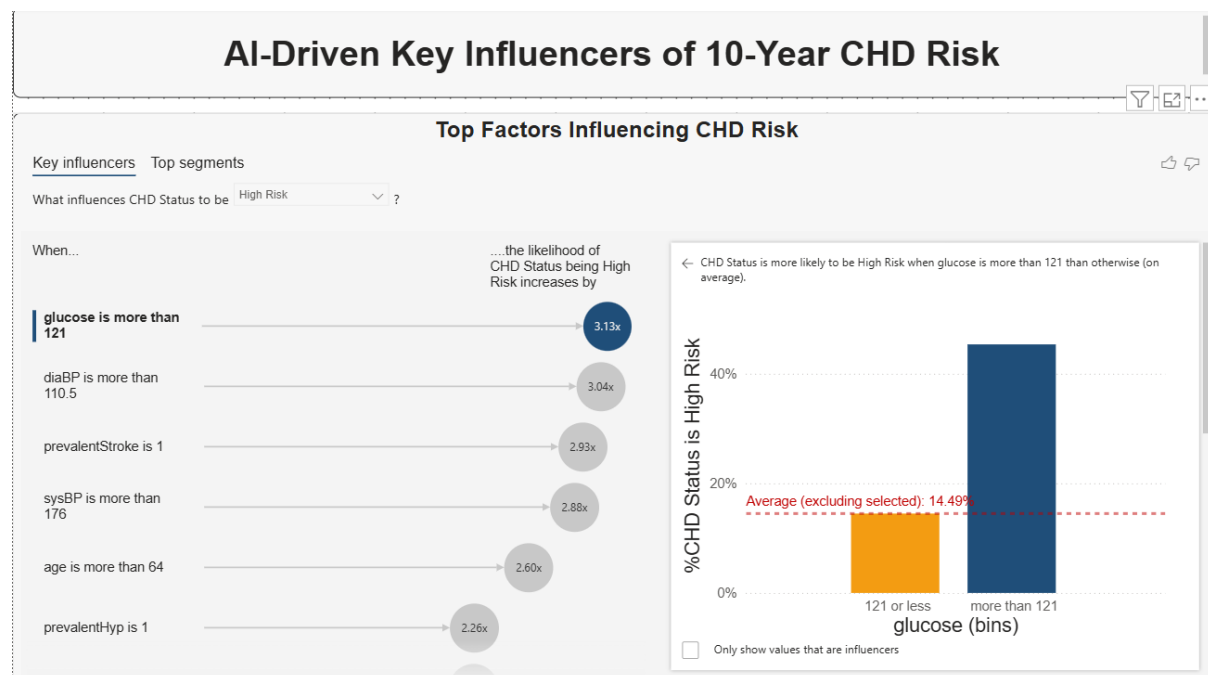
Clinical Indicators



1. **Blood pressure is a dominant clinical factor:** Clear upward trend—higher `sysBP` and `diaBP` correspond to more high-risk (red) cases → aligns with earlier feature importance.
2. **Cholesterol shows threshold-like behavior:** Risk increases gradually but spikes at very high levels (400+) → indicates nonlinear effect rather than simple linear increase.
3. **Diabetes chart is misleading (critical issue):** Both diabetic and non-diabetic show ~100% risk → likely a plotting/aggregation error (using counts instead of normalized probabilities). You must fix this before drawing conclusions.
4. **Key clinical takeaway:** Among reliable signals, **blood pressure > cholesterol > diabetes (uncertain due to error)** → BP control should be primary intervention target.

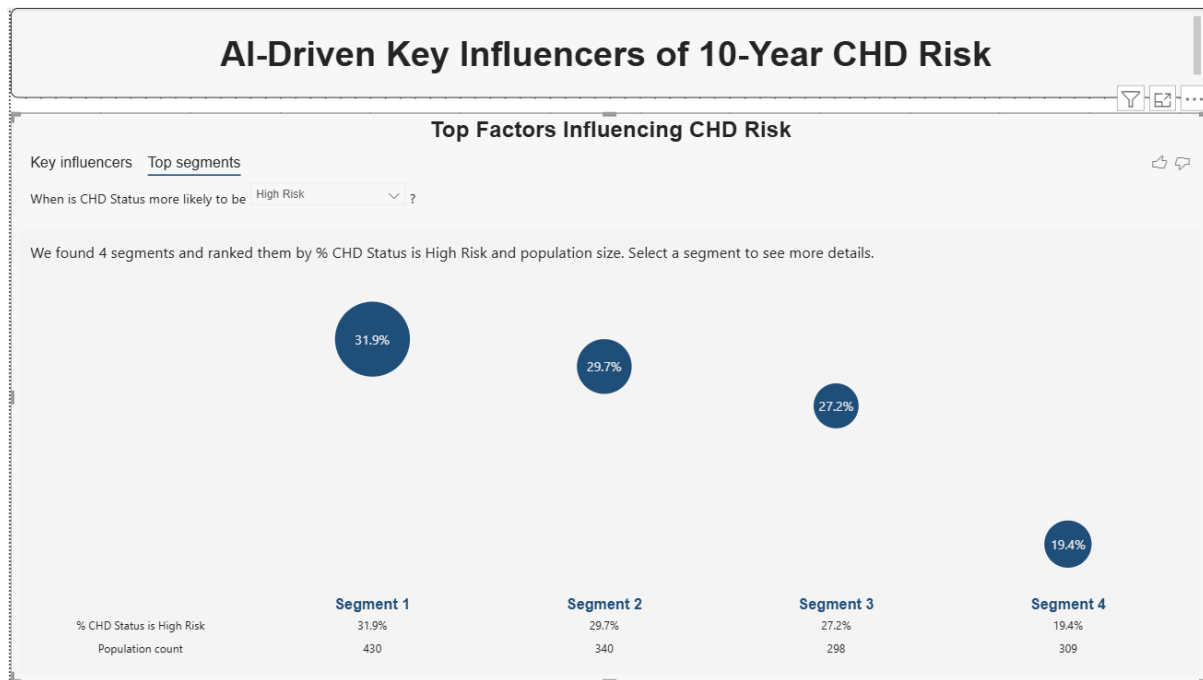
AI Driven Analysis

Key Infulence



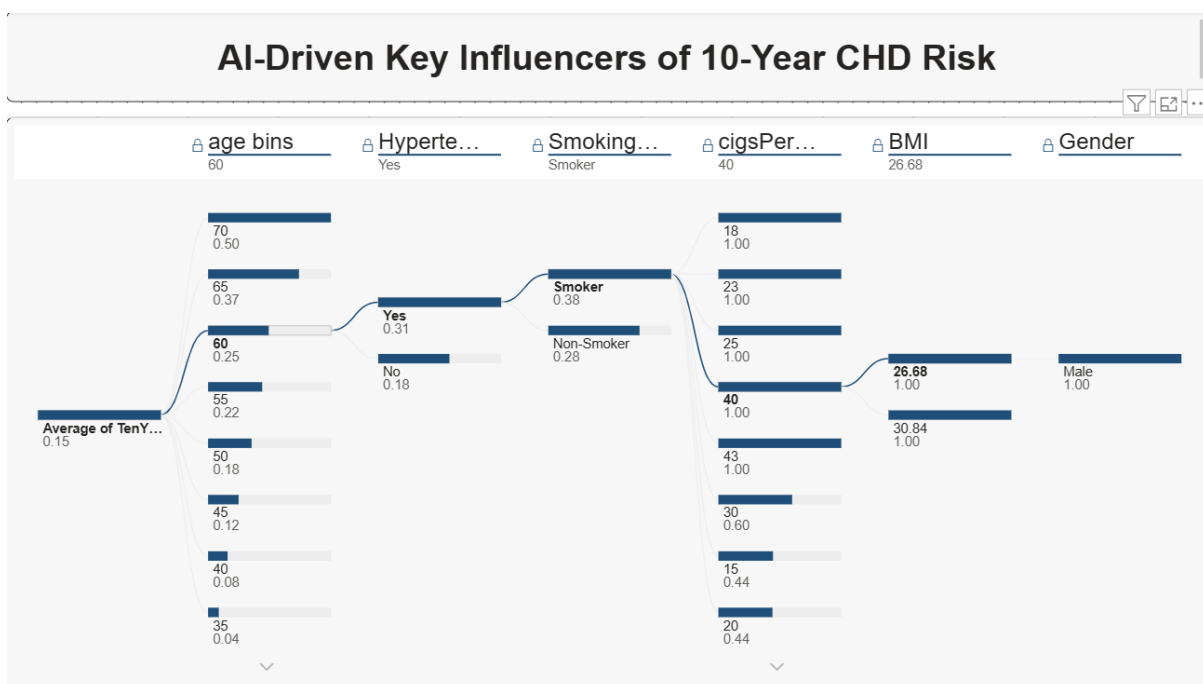
- Top driver identified: glucose > 121** → increases CHD risk by ~3.13x → strongest threshold-based predictor (nonlinear effect clearly captured).
- Blood pressure still critical:** diaBP > 110.5 (~3.04x) and sysBP > 176 (~2.88x) → confirms hypertension as a major risk amplifier.
- High-risk clinical history matters:** prevalentStroke = 1 (~2.93x) and prevalentHyp = 1 (~2.26x) → past conditions significantly elevate future CHD risk.
- Age acts as a multiplier:** age > 64 (~2.60x) → not just linear increase, but sharp rise after threshold → important for risk stratification.
- Key insight (very important):** These are **threshold effects**, not linear → risk jumps after certain values → tree-based/AI models capture this better than linear models.

Top Segments



1. **High-risk segments identified:** Segment 1 (31.9%) and Segment 2 (29.7%) have the highest CHD risk → priority groups for intervention.
2. **Not just risk, but size matters:** Segment 1 has the largest population (430) → **highest overall impact** if targeted.
3. **Gradual risk drop across segments:** Risk decreases from ~32% → ~19% → shows clear stratification of population into risk tiers.
4. **Actionable insight:** Focus on **Segment 1 & 2** for screening/prevention (high risk + significant population), while Segment 4 represents lower priority.

Decision Tree



1. **High-risk profile is multi-factor, not single:** The pathway shows risk increasing step-by-step → **age (~60+) + hypertension + smoking + high cigs/day (~40) + higher BMI + male** = highest CHD risk segment.
2. **Age is the entry gate:** Risk jumps significantly after ~60 → acts as the primary filter before other factors amplify risk.
3. **Compounding effect (critical insight):** Each factor adds risk sequentially (not independently) → e.g., smoker alone ≠ high risk, but smoker + hypertensive + older = sharply elevated risk.
4. **Smoking intensity matters again:** Path specifically highlights ~40 cigarettes/day → confirms earlier finding that **quantity > status**.
5. **Clinical takeaway (very important):**
Highest priority patients = **older (60+) + hypertensive + heavy smokers** → this is your **target cohort for intervention, screening, and prevention programs**.

Resources and References:

- <https://lms.simplilearn.com/courses/4211/Power-BI-for-Beginners>